

KI-unterstützte Erstellung von ETL-Prozessen - Eine experimentelle Evaluierung

Bachelorarbeit

zur Erlangung des Grades

Bachelor of Science

der Fachrichtung Informatik

an der Dualen Hochschule Baden-Württemberg (DHBW) Stuttgart

von

Luis Geiger

02.05.2026

Matrikelnummer:

9144849

Kurs:

TINF23CE

Gutachter an der DHBW Stuttgart: Prof. Dr. Name

Erklärung

Ich erkläre mich hiermit ehrenwörtlich, dass ich die vorliegende Arbeit Bachelorarbeit mit dem Thema „*KI-unterstützte Erstellung von ETL-Prozessen - Eine experimentelle Evaluierung*“ selbständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Aus den benutzten Quellen, direkt oder indirekt, übernommene Gedanken habe ich als solche kenntlich gemacht. Diese Arbeit wurde bisher in gleicher oder ähnlicher Form oder auszugsweise noch keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Stuttgart, 02.05.2026

Ort, Datum

L.Geiger

Luis Geiger

Contents

Abkürzungsverzeichnis	V
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielsetzung und Forschungsfragen	2
1.3 Methodisches Vorgehen	3
1.4 Aufbau der Arbeit	4
2 Grundlagen	4
2.1 ETL-Prozesse	5
2.1.1 Definition und Phasen	5
2.1.2 Typische Herausforderungen	6
2.1.3 Klassische Werkzeuge und Verfahren	6
2.2 Datenqualität und Datenbereinigung	7
2.2.1 Dimensionen der Datenqualität	7
2.2.2 Häufige Datenprobleme	8
2.3 Künstliche Intelligenz und maschinelles Lernen	8
2.4 Large Language Models	9
2.4.1 Funktionsweise und Architektur	9
2.4.2 Überblick aktueller Modelle	10
2.4.3 Stärken und Limitationen bei strukturierten Daten	10
2.5 Prompt Engineering	11
2.5.1 Grundlegende Prompting-Strategien	11
2.5.2 Einflussfaktoren auf die Ergebnisqualität	12
3 Stand der Forschung	12
3.1 KI-Einsatz in der Datenverarbeitung	13
3.2 LLMs für strukturierte Daten und Tabellen	13
3.3 Verwandte Arbeiten zur Automatisierung von ETL	14
3.4 Forschungslücke und Abgrenzung	15
4 Konzeption der experimentellen Evaluierung	16
4.1 Forschungsdesign und Hypothesen	16
4.2 Auswahl der Large Language Models	19
4.3 Definition der ETL-Aufgaben	20
4.3.1 Datenbereinigung	20
4.3.2 Datentransformation	21

4.3.3	Duplikaterkennung	21
4.4	Prompting-Strategien	22
4.5	Evaluationskriterien	22
4.5.1	Genauigkeit	23
4.5.2	Vollständigkeit	23
4.5.3	Konsistenz	23
4.5.4	Effizienz	24
4.6	Vergleich gegen eine manuelle ETL-Pipeline	24
5	Erstellung des synthetischen Datensatzes	25
5.1	Anforderungen an den Datensatz	25
5.2	Datenmodell und -struktur	26
5.3	Eingebaute Datenqualitätsprobleme	27
5.4	Generierungsverfahren und verwendete Werkzeuge	29
5.5	Validierung des Datensatzes	30
6	Implementierung	31
6.1	Systemarchitektur und Versuchsumgebung	31
6.2	Eingesetzte Technologien und Bibliotheken	32
6.3	Umsetzung der klassischen ETL-Pipeline	33
6.4	Integration der LLMs in die Pipeline	34
6.5	Umsetzung der Prompting-Strategien	35
6.6	Mess- und Auswertungsverfahren	36
7	Durchführung der Experimente	37
7.1	Versuchsablauf	37
7.2	Erhobene Messdaten	38
7.3	Reproduzierbarkeit und Versuchsbedingungen	39
8	Ergebnisse und Auswertung	42
8.1	Ergebnisse je ETL-Aufgabe	42
8.2	Vergleich der Modelle	42
8.3	Vergleich der Prompting-Strategien	42
8.4	Vergleich KI-basiert vs. klassisch	42
8.5	Diskussion der Ergebnisse	42
8.5.1	Stärken des LLM-Einsatzes	42
8.5.2	Schwächen und Risiken	42
8.5.3	Einordnung in den Stand der Forschung	42

9 Handlungsempfehlungen für die Praxis	42
9.1 Eignung von LLMs für unterschiedliche ETL-Szenarien	42
9.2 Empfehlungen zur Modell- und Prompt-Auswahl	42
9.3 Hybride Architekturen	42
10 Fazit und Ausblick	42
10.1 Zusammenfassung der Ergebnisse	42
10.2 Beantwortung der Forschungsfragen	42
10.3 Limitationen der Arbeit	42
10.4 Ausblick auf weitere Forschung	42
Bibliography	43

Abkürzungsverzeichnis

ETL Extract, Transform, Load

LLM Large Language Model

KI Künstliche Intelligenz

ML maschinelles Lernen

1. Einleitung

1.1 Motivation und Problemstellung

Die Verfügbarkeit, Qualität und zeitnahe Bereitstellung von Daten sind zu einem zentralen Erfolgsfaktor für datengetriebene Organisationen geworden. Im Zentrum der dafür notwendigen Dateninfrastruktur stehen seit Jahrzehnten Extract, Transform, Load (ETL)-Prozesse, die Daten aus heterogenen Quellsystemen extrahieren, in eine einheitliche Struktur überführen und in Zielsysteme wie Data Warehouses oder Analyseplattformen laden [1, 2]. Trotz ihrer langen Etablierung gelten klassische ETL-Pipelines zunehmend als Engpass: Sie sind in der Regel regelbasiert, statisch modelliert und erfordern erheblichen manuellen Aufwand bei Entwurf, Anpassung und Wartung [3, 4]. Mit dem Zuwachs von Datenvolumen, der Heterogenität der Quellen und steigenden Anforderungen an Aktualität geraten herkömmliche ETL-Architekturen an ihre Grenzen [2, 5]. Parallel dazu haben sich in den letzten Jahren Large Language Models (LLMs) als leistungsfähige Werkzeuge etabliert, die neben natürlicher Sprache auch strukturierte und semi-strukturierte Daten verarbeiten können [6]. Arbeiten zeigen, dass LLMs im Kontext der Datenverarbeitung Aufgaben wie Datenbereinigung, Schema-Mapping, Transformationslogik oder Anomalieerkennung übernehmen oder zumindest unterstützen können [7–9]. Damit eröffnet sich die Perspektive, klassische ETL-Prozesse durch Künstliche Intelligenz (KI)-gestützte Komponenten zu ergänzen oder partiell zu ersetzen, um manuellen Aufwand zu reduzieren und die Anpassungsfähigkeit der Pipelines zu erhöhen. Der praktische Anreiz liegt dabei weniger in der technischen Machbarkeit an sich als vielmehr in der Aussicht auf Effizienz- und Kostenvorteile: Übernehmen Modelle Routineschritte der Transformationslogik ganz oder unterstützend, lässt sich der zeitintensive und damit kostenträchtige manuelle Entwicklungs- und Wartungsaufwand spürbar verringern, was den Einsatz gerade im wirtschaftlichen Kontext motiviert. Gleichzeitig ist die praktische Eignung von LLMs für ETL-Aufgaben bislang nur unzureichend systematisch untersucht. Während konzeptionelle Beiträge die potenziellen Vorteile beschreiben, fehlt es an empirischen Vergleichen, die unterschiedliche Modelle und Prompting-Strategien unter kontrollierten Bedingungen gegen klassische Verfahren stellen. Offen ist insbesondere, wie zuverlässig LLMs typische ETL-Teilaufgaben wie Datenbereinigung, Datentransformation und Duplikaterkennung bearbeiten, wie stabil und reproduzierbar ihre Ergebnisse aufgrund der probabilistischen Modellarchitektur ausfallen, wie sich verschiedene Prompting-Ansätze auf die Ergebnisqualität auswirken und unter welchen Bedingungen ein Einsatz gegenüber

etablierten regelbasierten Methoden sinnvoll ist. Offen bleibt zudem, wie sich ein solcher Einsatz zum heutigen De-facto-Standard der Softwareentwicklung verhält, bei dem KI nicht autonom, sondern als Assistenz einer menschlichen Fachkraft wirkt: Inwieweit die Modelle den manuellen Aufwand tatsächlich verringern, statt ihn lediglich zu verlagern, ist bislang kaum systematisch beleuchtet. Genau an diesem Forschungsdefizit setzt die vorliegende Arbeit an.

1.2 Zielsetzung und Forschungsfragen

Ziel dieser Arbeit ist eine experimentelle Evaluierung, inwieweit sich aktuelle LLMs für ausgewählte ETL-Aufgaben eignen – sei es als eigenständige Automatisierung oder als unterstützendes Werkzeug einer menschlichen Fachkraft. Im Zentrum steht der systematische Vergleich mehrerer aktueller LLMs und unterschiedlicher Prompting-Strategien, bewertet anhand von Genauigkeit, Vollständigkeit, Konsistenz und Effizienz bei der Lösung der definierten ETL-Teilaufgaben und ausdrücklich nicht als allgemeiner Leistungsvergleich der Modelle. Grundlage ist ein kontrolliert erstellter synthetischer Datensatz, der typische Datenqualitätsprobleme abbildet. Die Ergebnisse werden mit einer klassisch implementierten ETL-Pipeline verglichen, um Stärken, Schwächen und praktische Einsatzgrenzen KI-gestützter Ansätze fundiert einordnen zu können. Auf dieser Grundlage sollen Handlungsempfehlungen für den Einsatz von LLMs in realen ETL-Szenarien abgeleitet werden. Aus dieser Zielsetzung ergeben sich zwei leitende Forschungsfragen:

FF1: Inwieweit eignen sich aktuelle LLMs für die Bearbeitung typischer ETL-Teilaufgaben – Datenbereinigung, Datentransformation und Duplikaterkennung – im Vergleich zu einer klassischen, regelbasierten Pipeline, und unter welchen Bedingungen ist je nach Aufgabentyp ein klassischer, KI-gestützter oder hybrider Ansatz zu empfehlen?

FF2: Wie stark beeinflusst die Wahl der Prompting-Strategie die Ergebnisqualität der untersuchten LLMs, und welche Strategien sind für die betrachteten ETL-Aufgaben zu empfehlen?

Da LLMs probabilistische Ausgaben erzeugen, wird die Reproduzierbarkeit ihrer Ergebnisse nicht als eigenständige Forschungsfrage, sondern als querschnittliche Untersuchungsdimension über beide Fragen hinweg behandelt: Jede Konfiguration wird mehrfach ausgeführt, sodass die Stabilität der Ergebnisse durchgängig mitbewertet

werden kann.

1.3 Methodisches Vorgehen

Zur Beantwortung der formulierten Forschungsfragen wird ein mehrstufiges, experimentell ausgerichtetes Vorgehen gewählt, das konzeptionelle, gestalterische und empirische Elemente miteinander verbindet. Den Ausgangspunkt bildet eine strukturierte Aufarbeitung der theoretischen Grundlagen zu ETL-Prozessen, Datenqualität, LLMs und Prompt Engineering sowie eine Analyse des aktuellen Forschungsstands zum Einsatz von KI in der Datenintegration. Diese literaturbasierte Vorarbeit dient dazu, relevante Konzepte, etablierte Verfahren und bestehende Forschungslücken zu identifizieren und die anschließende Evaluierung theoretisch zu fundieren. Auf dieser Grundlage wird in einem zweiten Schritt das experimentelle Design entwickelt. Dazu zählen die Auswahl mehrerer aktueller LLMs, die Definition der zu untersuchenden ETL-Teilaufgaben Datenbereinigung, Datentransformation und Duplikaterkennung sowie die Festlegung unterschiedlicher Prompting-Strategien. Ergänzend werden Evaluationskriterien operationalisiert, anhand derer die Ergebnisse aller Verfahren vergleichbar gemessen werden können. Da LLMs aufgrund ihrer probabilistischen Natur nicht-deterministische Ausgaben erzeugen können, wird zusätzlich die Reproduzierbarkeit der Ergebnisse als eigenständige Untersuchungsdimension berücksichtigt. Als Vergleichsmaßstab dient eine klassische, regelbasiert implementierte ETL-Pipeline. Im dritten Schritt wird ein synthetischer Datensatz erstellt, der typische Datenqualitätsprobleme realer Datenbestände kontrolliert abbildet. Die gezielte Konstruktion erlaubt es, die Eigenschaften der Eingangsdaten exakt zu kennen und damit reproduzierbare sowie aussagekräftige Vergleiche zwischen den untersuchten Ansätzen zu ermöglichen. Anschließend erfolgt die technische Umsetzung der Versuchsumgebung, in der sowohl die klassische ETL-Pipeline als auch die LLM-basierten Verarbeitungsschritte implementiert und über einheitliche Schnittstellen ausgeführt werden. Die eigentliche Durchführung der Experimente erfolgt unter standardisierten Bedingungen, wobei für jede Kombination aus Modell, Prompting-Strategie und ETL-Aufgabe Messdaten zu Qualität, Stabilität und Effizienz erhoben werden. Um die Reproduzierbarkeit der LLM-Ergebnisse beurteilen zu können, wird jede Konfiguration mehrfach ausgeführt. Die Auswertung kombiniert eine quantitative Analyse der definierten Kennzahlen mit einer qualitativen Einordnung beobachteter Stärken und Schwächen. Aus den Ergebnissen werden abschließend Handlungsempfehlungen abgeleitet, die eine fundierte Einschätzung über den sinnvollen Einsatz von LLMs in unterschiedlichen ETL-Szenarien ermöglichen.

1.4 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in zehn Kapitel. Nach der vorliegenden Einleitung, in der Motivation, Zielsetzung, Forschungsfragen und methodisches Vorgehen dargelegt werden, behandelt Kapitel 2 die theoretischen Grundlagen. Hier werden zentrale Konzepte zu ETL-Prozessen, Datenqualität, LLMs sowie Prompt Engineering eingeführt, die als Bezugsrahmen für die weitere Untersuchung dienen. Kapitel 3 widmet sich dem aktuellen Stand der Forschung. Es werden Arbeiten zum Einsatz von KI in der Datenverarbeitung, zur Verarbeitung strukturierter Daten durch LLMs sowie zur Automatisierung von ETL-Prozessen aufgearbeitet und gegeneinander abgegrenzt. Auf dieser Basis wird die für diese Arbeit relevante Forschungslücke präzisiert. In Kapitel 4 wird die Konzeption der experimentellen Evaluierung beschrieben. Dies umfasst das Forschungsdesign, die Auswahl der untersuchten Modelle, die Definition der ETL-Aufgaben, die eingesetzten Prompting-Strategien sowie die Evaluationskriterien einschließlich der manuellen Pipeline. Kapitel 5 dokumentiert anschließend die Erstellung des synthetischen Datensatzes, einschließlich Anforderungen, Datenmodell, eingebauter Qualitätsprobleme und der durchgeführten Validierung. Die technische Umsetzung wird in Kapitel 6 dargestellt. Es beschreibt die Systemarchitektur, die eingesetzten Technologien sowie die konkrete Implementierung der klassischen ETL-Pipeline und der LLM-basierten Komponenten. Kapitel 7 behandelt die Durchführung der Experimente und legt Versuchsablauf, erhobene Messdaten sowie die Bedingungen zur Sicherstellung der Reproduzierbarkeit offen. Kapitel 8 präsentiert und analysiert die Ergebnisse. Neben einer aufgabenspezifischen Auswertung erfolgen ein Vergleich der Modelle, der Prompting-Strategien sowie eine Gegenüberstellung von KI-basiertem und klassischem Ansatz. Die Ergebnisse werden anschließend diskutiert und in den Stand der Forschung eingeordnet. Aus den gewonnenen Erkenntnissen leitet Kapitel 9 Handlungsempfehlungen für die Praxis ab, insbesondere zur Eignung von LLMs in unterschiedlichen ETL-Szenarien sowie zur Gestaltung hybrider Architekturen. Kapitel 10 fasst die zentralen Ergebnisse zusammen, beantwortet die in Abschnitt 1.2 formulierten Forschungsfragen, reflektiert die Limitationen der Arbeit und gibt einen Ausblick auf weiterführende Forschungsfragen.

2. Grundlagen

Dieses Kapitel legt die theoretischen Grundlagen, auf denen die experimentelle Evaluierung der folgenden Kapitel aufbaut. Zunächst werden ETL-Prozesse als etabliertes Ver-

fahren der Datenintegration beschrieben (Abschnitt 2.1). Anschließend wird das Konzept der Datenqualität als zentraler Bezugspunkt für die untersuchten ETL-Teilaufgaben eingeführt (Abschnitt 2.2). Darauf folgt eine Einordnung von Künstlicher Intelligenz und maschinellem Lernen als übergeordnetem Rahmen (Abschnitt 2.3), bevor Large Language Models als die in dieser Arbeit eingesetzte verarbeitende Komponente betrachtet werden (Abschnitt 2.4). Den Abschluss bildet eine Darstellung des Prompt Engineering als zentrale Steuerungsmethode für diese Modelle (Abschnitt 2.5).

2.1 ETL-Prozesse

2.1.1 Definition und Phasen

Der Begriff ETL steht für die drei aufeinanderfolgenden Phasen *Extract*, *Transform* und *Load* und bezeichnet einen der zentralen Prozesse der Datenintegration, mit dem Daten aus unterschiedlichen Quellsystemen in ein einheitliches Zielsystem überführt werden [1, 2]. Historisch ist der ETL-Prozess eng mit dem Aufbau von Data Warehouses verbunden, in denen konsolidierte und bereinigte Daten als Grundlage für Auswertungen und Entscheidungsunterstützung bereitgestellt werden [1, 10]. Der Prozess bildet damit das Bindeglied zwischen unverarbeiteten Rohdaten und den daraus abgeleiteten, nutzbaren Informationen [1, 2].

In der ersten Phase, der Extraktion, werden die relevanten Daten aus einer oder mehreren Quellen ausgelesen. Diese Quellen sind in der Praxis heterogen und reichen von relationalen Datenbanken über Dateien und Tabellen bis hin zu Schnittstellen externer Dienste, sodass bereits hier unterschiedliche Formate und Strukturen zusammengeführt werden müssen [1, 2]. Die Transformationsphase bildet den inhaltlichen Kern des Prozesses: Die extrahierten Daten werden bereinigt, vereinheitlicht und in die vom Zielsystem geforderte Struktur überführt. Typische Operationen sind die Korrektur fehlerhafter oder fehlender Werte, die Vereinheitlichung von Formaten, die Zusammenführung und Aggregation von Datensätzen, das Auflösen von Duplikaten sowie das Abbilden der Quellstrukturen auf das Zielschema [1, 2, 5]. In der abschließenden Ladephase werden die transformierten Daten schließlich in das Zielsystem geschrieben, etwa in ein Data Warehouse oder eine Analyseplattform, wobei zwischen vollständigem Laden und inkrementellem Aktualisieren unterschieden wird [1, 10].

Die strikte Abfolge der drei Phasen ist dabei nicht das einzig mögliche Vorgestaltungsmuster. Insbesondere im Kontext von Big Data hat sich mit ELT (Extract, Load,

Transform) eine Variante etabliert, bei der die Rohdaten zunächst geladen und erst innerhalb des Zielsystems transformiert werden, um von dessen Verarbeitungskapazitäten zu profitieren [2, 5]. Für die vorliegende Arbeit bleibt jedoch die klassische ETL-Reihenfolge maßgeblich, da der Schwerpunkt auf den Transformationsschritten der Datenbereinigung, Datentransformation und Duplikaterkennung liegt.

2.1.2 Typische Herausforderungen

Obwohl ETL-Prozesse seit Jahrzehnten etabliert sind, gelten sie in modernen Datenlandschaften zunehmend als anspruchsvoll und wartungsintensiv. Diese Einschätzung gründet auf mehreren miteinander verbundenen Herausforderungen. Eine zentrale davon ist die Sicherung der Datenqualität: Da die Eingangsdaten aus unterschiedlichen Quellen stammen, weisen sie häufig Fehler, Lücken und Inkonsistenzen auf, die im Transformationsschritt erkannt und behoben werden müssen, um verlässliche Analyseergebnisse zu gewährleisten [1, 11]. Eng damit verbunden ist die Heterogenität der Quellen, die sich in unterschiedlichen Formaten, Schemata und Bedeutungen gleichartiger Felder äußert und einen erheblichen Aufwand bei der Vereinheitlichung verursacht [1, 2].

Hinzu kommt die Frage der Skalierbarkeit. Mit dem stetigen Wachstum von Datenvolumen und -vielfalt geraten klassische, batch-orientierte ETL-Architekturen an ihre Grenzen und können den Anforderungen an Verarbeitungsgeschwindigkeit und Aktualität nur noch eingeschränkt gerecht werden [2, 5]. Ein weiterer kritischer Aspekt ist der hohe manuelle Aufwand bei Entwurf, Anpassung und Wartung: Klassische Pipelines sind in der Regel regelbasiert und statisch modelliert, sodass jede Änderung an den Quellstrukturen oder den fachlichen Anforderungen einen manuellen Eingriff erfordert [3, 4]. Diese geringe Anpassungsfähigkeit führt dazu, dass ein Großteil der Arbeitszeit von Datenfachleuten in die Erstellung und Pflege solcher Pipelines fließt, was sie zu einem häufig benannten Engpass datengetriebener Organisationen macht [3, 5]. Genau diese Schwachstellen motivieren die in dieser Arbeit untersuchte Frage, inwieweit lernende und sprachbasierte Verfahren den manuellen Aufwand reduzieren und die Flexibilität klassischer ETL-Prozesse erhöhen können.

2.1.3 Klassische Werkzeuge und Verfahren

Zur Umsetzung von ETL-Prozessen hat sich über die Jahre ein breites Spektrum an Werkzeugen und Verfahren herausgebildet. Auf der einen Seite stehen dedizierte

ETL-Plattformen mit grafischer Modellierung wie Talend, Pentaho oder Informatica, die den Prozess über vorkonfigurierte Komponenten abbilden und sich vor allem in betrieblichen Data-Warehouse-Umgebungen etabliert haben [1, 10]. Auf der anderen Seite werden ETL-Strecken zunehmend programmatisch umgesetzt, etwa mit der Python-Bibliothek Pandas für die Verarbeitung tabellarischer Daten oder mit verteilten Frameworks wie Apache Spark für große Datenmengen [1, 2].

Allen klassischen Ansätzen ist gemeinsam, dass die Verarbeitungslogik regelbasiert und explizit vorgegeben wird: Entwicklerinnen und Entwickler definieren die Transformations- und Bereinigungsregeln manuell, etwa in Form von Bedingungen, Zuordnungstabellen oder Skripten [3, 10]. Dieser Ansatz bietet den Vorteil hoher Nachvollziehbarkeit und deterministischer, reproduzierbarer Ergebnisse, stößt jedoch dort an Grenzen, wo Datenprobleme vielfältig, kontextabhängig oder schwer im Voraus zu formalisieren sind [3, 4]. In der vorliegenden Arbeit dient eine solche regelbasiert implementierte Pipeline als Vergleichsmaßstab, gegen den die untersuchten LLM-gestützten Verfahren gemessen werden.

2.2 Datenqualität und Datenbereinigung

2.2.1 Dimensionen der Datenqualität

Datenqualität ist ein mehrdimensionales Konstrukt, das beschreibt, inwieweit ein Datenbestand für seinen vorgesehenen Verwendungszweck geeignet ist. In der Literatur hat sich ein Kern wiederkehrender Qualitätsdimensionen herausgebildet, zu denen insbesondere Genauigkeit, Vollständigkeit, Konsistenz, Aktualität und Eindeutigkeit zählen [11, 12]. Die Genauigkeit beschreibt, inwieweit die gespeicherten Werte die reale Sachlage korrekt abbilden, während die Vollständigkeit den Grad angibt, in dem alle erforderlichen Werte tatsächlich vorhanden sind [11, 12]. Die Konsistenz bezieht sich auf die Widerspruchsfreiheit der Daten innerhalb eines Bestands und über Systemgrenzen hinweg, etwa bei einheitlichen Formaten und Bezeichnungen [11, 12]. Die Aktualität erfasst, ob die Daten den jeweils gültigen Zeitstand widerspiegeln, und die Eindeutigkeit verlangt, dass jeder reale Sachverhalt nur einmal und nicht in mehreren widersprüchlichen oder duplizierten Datensätzen abgelegt ist [11, 12].

Mit dem Aufkommen von Big Data ist dieser klassische Kanon erweitert worden, da Volumen, Geschwindigkeit und Vielfalt der Daten zusätzliche Anforderungen stellen und feinere Differenzierungen der Qualitätsdimensionen erforderlich machen [11, 12]. Für

die vorliegende Arbeit sind insbesondere die Dimensionen Genauigkeit, Vollständigkeit, Konsistenz und Eindeutigkeit von Bedeutung, da sie unmittelbar mit den untersuchten ETL-Teilaufgaben der Datenbereinigung, Datentransformation und Duplikaterkennung korrespondieren.

2.2.2 Häufige Datenprobleme

Aus den Qualitätsdimensionen lassen sich die in der Praxis am häufigsten auftretenden Datenprobleme ableiten, die im Rahmen der Datenbereinigung erkannt und behoben werden müssen. Fehlende Werte gehören zu den verbreitetsten Mängeln und beeinträchtigen unmittelbar die Vollständigkeit; sie erfordern Entscheidungen über Imputation, Entfernung oder Kennzeichnung der betroffenen Datensätze [11, 13]. Inkonsistente Formate – etwa unterschiedliche Datums-, Zahlen- oder Einheitendarstellungen sowie abweichende Schreibweisen gleicher Inhalte – beeinträchtigen die Konsistenz und müssen im Transformationsschritt vereinheitlicht werden [11, 12].

Ein weiteres zentrales Problem sind Duplikate, also mehrfach vorhandene Datensätze, die denselben realen Sachverhalt beschreiben, sich aber in Schreibweise oder Vollständigkeit unterscheiden können. Ihre Erkennung ist anspruchsvoll, da sie über den exakten Abgleich hinaus häufig eine unscharfe Ähnlichkeitsbetrachtung erfordert [13, 14]. Hinzu kommen Ausreißer und fehlerhafte Werte, die durch Eingabefehler, Messfehler oder Systembrüche entstehen und die Genauigkeit des Datenbestands gefährden [11, 13]. Die Bedeutung einer gründlichen Bereinigung ergibt sich daraus, dass mangelhafte Eingangsdaten unmittelbar auf nachgelagerte Analysen und Modelle durchschlagen, was häufig mit dem Grundsatz “garbage in, garbage out” umschrieben wird [11, 12]. Die gezielte Konstruktion solcher Datenprobleme bildet später die Grundlage des in Kapitel 5 beschriebenen synthetischen Datensatzes.

2.3 Künstliche Intelligenz und maschinelles Lernen

Die in dieser Arbeit untersuchten Sprachmodelle sind Teil eines übergeordneten Forschungs- und Anwendungsfeldes, das dieser Abschnitt einordnet, bevor die Modelle selbst betrachtet werden. Unter dem Begriff der KI wird das Bestreben verstanden, Maschinen mit Fähigkeiten auszustatten, die üblicherweise menschliche Intelligenz voraussetzen – etwa das Erkennen von Mustern, das Ziehen von Schlüssen oder das Treffen von Entscheidungen [15, 16]. KI ist damit ein weit gefasster Oberbegriff, der von regelbasierten Expertensystemen bis zu lernenden Verfahren reicht.

Den heute dominierenden Zweig der KI bildet maschinelles Lernen (ML). Anders als klassische, regelbasierte Systeme, deren Verhalten explizit durch Entwickelnde vorgegeben wird, leiten ML-Verfahren die zugrunde liegenden Zusammenhänge selbstständig aus Beispieldaten ab und übertragen sie auf neue, ungesehene Fälle [16, 17]. Genau dieser Unterschied – gelernte statt fest programmierter Verarbeitungslogik – ist für die vorliegende Arbeit zentral, da er den Kontrast zwischen den untersuchten LLMs und der klassischen, regelbasierten Vergleichspipeline begründet.

Im Kontext der Datenverarbeitung sind ML-Verfahren bereits etabliert und werden unter anderem zur Datenbereinigung, zur Imputation fehlender Werte, zum Abgleich von Entitäten und zur Anomalieerkennung eingesetzt [13, 17]. Eine besonders leistungsfähige Ausprägung des maschinellen Lernens ist das auf tiefen neuronalen Netzen beruhende *Deep Learning*, dessen Fortschritte – insbesondere die Transformer-Architektur – die Grundlage der im folgenden Abschnitt behandelten LLMs bilden [6, 18].

2.4 Large Language Models

2.4.1 Funktionsweise und Architektur

LLMs sind Sprachmodelle mit einer sehr großen Zahl an Parametern, die auf großen Mengen an Textdaten vortrainiert werden und in der Lage sind, natürliche Sprache zu verstehen und zu erzeugen [6, 18]. Technische Grundlage nahezu aller aktuellen Modelle ist die Transformer-Architektur, deren Selbstaufmerksamkeitsmechanismus es erlaubt, Abhängigkeiten zwischen weit auseinanderliegenden Textbestandteilen effizient zu erfassen [6, 18]. Eingaben werden dabei zunächst in kleinere Einheiten, sogenannte Token, zerlegt, die das Modell intern als numerische Vektoren verarbeitet; die Ausgabe entsteht, indem das Modell schrittweise das jeweils wahrscheinlichste nachfolgende Token vorhersagt [6].

Charakteristisch für LLMs ist der zweistufige Entstehungsprozess aus einem aufwendigen Vortraining auf großen, allgemeinen Datenmengen und einer anschließenden Anpassung, etwa durch Feinabstimmung oder durch ein Training mit menschlichem Feedback, das die Modelle an die Erwartungen der Nutzenden ausrichtet [6]. Ein zentraler Befund der Forschung ist, dass mit zunehmender Modell- und Datengröße neue Fähigkeiten auftreten, die kleinere Modelle nicht aufweisen. Dazu zählt insbesondere das In-Context-Lernen, also die Fähigkeit, eine Aufgabe allein anhand von Anweisungen und Beispielen im Eingabetext zu lösen, ohne dass die Modellparameter

verändert werden [6, 19]. Genau diese Eigenschaft bildet die Grundlage dafür, LLMs ohne aufgabenspezifisches Nachtraining für ETL-Teilaufgaben einzusetzen. Zugleich ist die Ausgabe der Modelle probabilistischer Natur: Da das nächste Token aus einer Wahrscheinlichkeitsverteilung gezogen wird, können identische Eingaben zu unterschiedlichen Ausgaben führen, was unmittelbare Folgen für die Reproduzierbarkeit der Ergebnisse hat [6].

2.4.2 Überblick aktueller Modelle

Seit der breiten öffentlichen Aufmerksamkeit für dialogfähige Modelle hat sich das Feld der LLMs rasant entwickelt und ausdifferenziert [6, 20]. Auf dem Markt konkurriert heute eine Reihe leistungsfähiger Modellfamilien, darunter die GPT-Modelle von OpenAI, die Claude-Modelle von Anthropic, die Gemini-Modelle von Google sowie offen verfügbare Modelle wie die Llama-Reihe [6, 20]. Diese Modelle unterscheiden sich nicht nur in ihrer Architektur und Parameterzahl, sondern auch in praxisrelevanten Eigenschaften wie der Größe des Kontextfensters, der Verarbeitungsgeschwindigkeit, den Nutzungskosten und der Verfügbarkeit als offenes oder geschlossenes Modell [6, 20].

Eine wesentliche Entwicklung der jüngeren Vergangenheit ist die Erweiterung hin zu multimodalen Modellen, die neben Text auch weitere Modalitäten wie Bilder verarbeiten können [6, 18]. Angesichts der hohen Innovationsgeschwindigkeit verschieben sich die Leistungsgrenzen zwischen den Modellen kontinuierlich, weshalb vergleichende Bewertungen anhand standardisierter Kennzahlen wie Qualität, Geschwindigkeit und Preis an Bedeutung gewonnen haben [20, 21]. Für die vorliegende Arbeit ist diese Heterogenität von unmittelbarer Relevanz, da sie die Auswahl mehrerer aktueller Modelle für den experimentellen Vergleich begründet und nahelegt, dass sich die Eignung für ETL-Aufgaben zwischen den Modellen unterscheiden kann.

2.4.3 Stärken und Limitationen bei strukturierten Daten

Im Kontext der Datenverarbeitung liegt die Stärke von LLMs in ihrer Flexibilität: Sie können semi- und unstrukturierte Inhalte interpretieren, kontextabhängige Zusammenhänge erfassen und über reine Mustererkennung hinaus auch implizites Wissen einbringen, ohne dass die zugrunde liegenden Regeln explizit vorgegeben werden müssen [6, 7]. Damit eignen sie sich potenziell für Aufgaben wie die Bereinigung uneinheitlicher Werte, die Vereinheitlichung von Formaten oder das Erkennen ähnlicher Datensätze,

die mit starren Regeln nur schwer abzudecken sind [7].

Diesen Stärken stehen jedoch erhebliche Limitationen gegenüber. Eine zentrale Einschränkung ist die Neigung zu Halluzinationen, also zur Erzeugung plausibel wirkender, aber sachlich falscher oder frei erfundener Ausgaben, was bei der Verarbeitung strukturierter Daten zu schwer erkennbaren Fehlern führen kann [6, 7]. Hinzu kommt die Begrenzung durch das Kontextfenster, die der Menge an Daten, die in einem einzelnen Verarbeitungsschritt übergeben werden kann, eine technische Obergrenze setzt [6]. Ferner fallen die Nutzungskosten und die Verarbeitungszeit ins Gewicht, die insbesondere bei großen Datenmengen zu einem begrenzenden Faktor werden [6, 20]. Schließlich erschwert die bereits beschriebene probabilistische Natur der Modelle die Reproduzierbarkeit, da wiederholte Ausführungen identischer Aufgaben voneinander abweichende Ergebnisse liefern können [6, 7]. Diese Spannung zwischen Flexibilität und Verlässlichkeit bildet einen der zentralen Ausgangspunkte und zugleich einen der Untersuchungsgegenstände der vorliegenden Arbeit.

2.5 Prompt Engineering

2.5.1 Grundlegende Prompting-Strategien

Da LLMs ihre Aufgaben primär über das In-Context-Lernen lösen, kommt der Gestaltung der Eingabe – dem sogenannten Prompt – eine entscheidende Bedeutung zu. Unter Prompt Engineering versteht man das gezielte Entwerfen und Optimieren solcher Eingaben, um das Modellverhalten zu steuern, ohne die Modellparameter selbst zu verändern [22, 23]. Die einfachste Strategie ist das Zero-Shot-Prompting, bei dem das Modell eine Aufgabe allein anhand einer Anweisung und ohne weitere Beispiele bearbeitet [19, 22]. Beim Few-Shot-Prompting werden der Anweisung hingegen einige beispielhafte Ein- und Ausgabepaare beigelegt, anhand derer das Modell das gewünschte Verhalten ableitet; dieser Ansatz verbessert die Ergebnisqualität bei vielen Aufgaben spürbar [19, 22].

Darüber hinaus haben sich anspruchsvollere Strategien etabliert. Beim Chain-of-Thought-Prompting wird das Modell angeleitet, seine Lösung in nachvollziehbaren Zwischenschritten zu entwickeln, was sich insbesondere bei mehrstufigen oder schlussfolgernden Aufgaben als wirksam erwiesen hat [22, 23]. Eine weitere verbreitete Technik ist das Role-Prompting, bei dem dem Modell eine bestimmte Rolle oder Perspektive zugewiesen wird, um Stil und Inhalt der Ausgabe zu lenken [22, 23]. Angesichts

der Vielzahl mittlerweile beschriebener Techniken weisen systematische Übersichten allerdings auf eine uneinheitliche Begriffsbildung und eine teils fragmentierte Forschungslage hin, was eine sorgfältige Auswahl und Definition der eingesetzten Strategien erforderlich macht [22, 23]. Für die vorliegende Arbeit werden daher klar abgegrenzte Prompting-Strategien definiert und systematisch gegeneinander verglichen.

2.5.2 Einflussfaktoren auf die Ergebnisqualität

Die Qualität der von einem LLM erzeugten Ausgabe hängt von einer Reihe von Einflussfaktoren ab, die über die reine Wahl der Prompting-Strategie hinausgehen. Beim Few-Shot-Prompting wirken sich Auswahl, Anzahl und Reihenfolge der mitgegebenen Beispiele unmittelbar auf das Ergebnis aus, sodass schon kleine Änderungen an den Beispielen die Modellantwort verschieben können [19, 23]. Auch die genaue Formulierung und Strukturierung der Anweisung beeinflusst die Ergebnisqualität erheblich, weshalb das Auffinden geeigneter Prompts häufig einen iterativen Prozess darstellt [22, 24].

Ein technisch bedeutsamer Parameter ist die Temperatur, die steuert, wie stark die Ausgabe von der wahrscheinlichsten Vorhersage abweicht: Niedrige Werte führen zu deterministischeren und stabileren, hohe Werte zu vielfältigeren, aber weniger vorher-sagbaren Antworten, was die Reproduzierbarkeit der Ergebnisse unmittelbar berührt [6, 23]. Zudem skaliert die Leistungsfähigkeit beim Prompting mit der Größe und Leistungsfähigkeit des zugrunde liegenden Modells, sodass dieselbe Strategie je nach Modell unterschiedlich gut wirkt [19, 23]. Schließlich ist mit längeren und beispiel-reicheren Prompts ein höherer Verbrauch an Token und damit ein höherer Aufwand verbunden, sodass zwischen Ergebnisqualität und Effizienz abzuwägen ist [23, 24]. Diese Faktoren begründen, warum in der vorliegenden Arbeit sowohl unterschiedliche Modelle als auch unterschiedliche Prompting-Strategien unter kontrollierten Bedingungen verglichen und ihre Ergebnisse mehrfach erhoben werden.

3. Stand der Forschung

Aufbauend auf den theoretischen Grundlagen ordnet dieses Kapitel die vorliegende Arbeit in den aktuellen Forschungsstand ein. Zunächst wird der breitere Einsatz von Künstlicher Intelligenz in der Datenverarbeitung betrachtet (Abschnitt 3.1). Anschließend wird die Forschung zur Verarbeitung strukturierter und tabellarischer Daten durch LLMs

aufgearbeitet (Abschnitt 3.2), bevor verwandte Arbeiten zur Automatisierung von ETL-Prozessen diskutiert werden (Abschnitt 3.3). Auf dieser Grundlage wird abschließend die Forschungslücke präzisiert, an der die vorliegende Arbeit ansetzt (Abschnitt 3.4).

3.1 KI-Einsatz in der Datenverarbeitung

Der Einsatz von Künstlicher Intelligenz und maschinellem Lernen hat die Datenverarbeitung in den vergangenen Jahren grundlegend verändert. Schon vor dem Aufkommen großer Sprachmodelle wurden Verfahren des maschinellen Lernens eingesetzt, um Schritte der Datenaufbereitung, der Mustererkennung und der Entscheidungsunterstützung zu automatisieren und so aus großen Datenbeständen Erkenntnisse zu gewinnen [15, 16]. Insbesondere im Bereich der Datenaufbereitung haben sich lernende Verfahren als nützlich erwiesen, etwa bei der Datenbereinigung, der Imputation fehlender Werte, dem Abgleich von Entitäten und der Erkennung von Ausreißern [13, 17]. Eine systematische Übersicht zeigt dabei ein wechselseitiges Verhältnis: Maschinelles Lernen wird einerseits zur Datenbereinigung genutzt, andererseits ist eine hohe Datenqualität Voraussetzung für die Leistungsfähigkeit der Modelle selbst [13, 17].

Mit der Verfügbarkeit dialogfähiger Sprachmodelle hat sich dieser Trend noch einmal verstärkt. Frühe Arbeiten beschreiben, wie Modelle wie ChatGPT Datenfachleute bei zahlreichen Teilschritten ihres Arbeitsablaufs unterstützen können, von der Datenbereinigung und Vorverarbeitung über die Modellbildung bis hin zur Interpretation der Ergebnisse [7, 16]. Gleichzeitig betonen diese Beiträge, dass der Einsatz solcher Modelle mit Herausforderungen wie Verzerrungen, mangelnder Nachvollziehbarkeit und Fragen der Verlässlichkeit verbunden ist und daher einer kritischen Begleitung bedarf [7, 15]. Insgesamt zeichnet die Literatur das Bild einer zunehmend KI-gestützten Datenverarbeitung, in der die Automatisierung von Routineaufgaben im Vordergrund steht, eine systematische empirische Bewertung der erzielten Qualität jedoch häufig noch aussteht [13, 17].

3.2 LLMs für strukturierte Daten und Tabellen

Während LLMs ursprünglich für die Verarbeitung natürlicher Sprache entwickelt wurden, hat sich die Forschung zunehmend ihrer Anwendung auf strukturierte und tabellarische Daten zugewandt. Hierbei zeigt sich, dass die Modelle prinzipiell in der Lage

sind, tabellarische Inhalte zu interpretieren, Zusammenhänge zwischen Feldern zu erfassen und auf dieser Basis Transformations- und Bereinigungsaufgaben zu unterstützen [7, 25]. Ein wichtiger Anwendungsstrang ist die semantische Interpretation komplexer Datenbeziehungen, bei der LLMs über den rein syntaktischen Abgleich hinaus den inhaltlichen Bezug zwischen Datensätzen erschließen und so etwa beim Abbilden unterschiedlicher Schemata helfen können [25, 26].

Ein zweiter, intensiv untersuchter Bereich ist die Erzeugung synthetischer Daten durch LLMs. Mehrere Übersichtsarbeiten dokumentieren, dass sich große Sprachmodelle eignen, um realistische tabellarische und textuelle Datensätze zu generieren, die etwa zur Datenanreicherung oder zum Ausgleich seltener Fälle dienen [27, 28]. Zugleich verweisen empirische Studien auf deutliche Grenzen: Die Eignung der erzeugten Daten variiert stark je nach Aufgabe, und insbesondere bei subjektiven oder komplexen Aufgabenstellungen bleibt die Qualität der synthetischen Daten hinter realen Daten zurück [27, 29]. Für tabellarische Daten im Speziellen weist die Literatur darauf hin, dass dieser Datentyp trotz seiner praktischen Bedeutung lange vernachlässigt wurde und eigene methodische Herausforderungen mit sich bringt [30, 31]. Diese Befunde sind für die vorliegende Arbeit in zweifacher Hinsicht relevant: Sie begründen einerseits die Verwendung eines synthetischen Datensatzes und mahnen andererseits zur empirischen Prüfung der tatsächlichen Verarbeitungsqualität, statt sich auf die grundsätzliche Fähigkeit der Modelle zu verlassen [29, 31]. Da eine solche empirische Prüfung im produktiven Einsatz jedoch nicht bei jeder Ausführung wiederholt werden kann, rückt zugleich die Stabilität der Ergebnisse über wiederholte Ausführungen in den Blick.

3.3 Verwandte Arbeiten zur Automatisierung von ETL

Die Automatisierung von ETL-Prozessen mit Methoden der Künstlichen Intelligenz ist Gegenstand einer wachsenden Zahl von Arbeiten. Ein erster, schon länger verfolgter Ansatz nutzt klassische Verfahren des maschinellen Lernens, um einzelne Schritte der Pipeline zu automatisieren, etwa das Abbilden von Schemata, die Erkennung von Anomalien und die Ableitung von Transformationsregeln aus den Daten [3, 26]. Darauf aufbauend beschreibt eine Reihe neuerer Beiträge umfassendere Architekturen, in denen KI-Komponenten auf allen Stufen des ETL-Prozesses eingebettet werden, um Aufgaben wie automatisierte Schemaerkennung, intelligente Qualitätssicherung und Leistungsoptimierung zu übernehmen [9, 32].

Ein wiederkehrendes Leitbild dieser Arbeiten ist die Vorstellung selbstheilender und adaptiver Datenpipelines, die Probleme proaktiv erkennen, eigenständig beheben und sich an veränderte Datenstrukturen anpassen, statt rein reaktiv auf Fehler zu reagieren [8, 26]. In diesem Zusammenhang werden zunehmend autonome KI-Agenten diskutiert, die Datenaufnahme, Bereinigung und Integration weitgehend selbstständig ausführen sollen [8, 33]. Mehrere Beiträge widmen sich dabei spezifischen Teilaufgaben, etwa der intelligenten Anomalieerkennung in ETL-Strecken, der automatisierten Qualitätssicherung oder der automatisierten Erzeugung von Merkmalen [14, 34]. Insbesondere wird betont, dass nicht ein vollständiger Ersatz, sondern eine hybride Verknüpfung von regelbasierten Systemen und KI-Modellen erstrebenswert sei, um Nachvollziehbarkeit und Kontrolle bei gleichzeitig höherer Flexibilität zu wahren [9, 32].

Arbeiten, die explizit generative Sprachmodelle für die ETL-Automatisierung einsetzen, sind hingegen noch vergleichsweise jung. Eine Untersuchung zur generativen KI in der ETL-Automatisierung zeigt, dass sich mit Hilfe von Einbettungs- und generativen Verfahren aus natürlichsprachlichen Anfragen passende ETL-Lösungen ableiten lassen, der manuelle Aufwand dadurch deutlich reduziert, jedoch nicht vollständig beseitigt wird [4, 35]. Insgesamt überwiegen in diesem Feld konzeptionelle und architektonische Beiträge, die das Potenzial KI-gestützter ETL-Prozesse beschreiben, während belastbare, vergleichende Evaluierungen einzelner Modelle und Verarbeitungsstrategien noch selten sind [36, 37].

3.4 Forschungslücke und Abgrenzung

Aus der Aufarbeitung des Forschungsstands lassen sich mehrere Lücken ableiten, an denen die vorliegende Arbeit ansetzt. Erstens beschreiben viele Beiträge das Potenzial KI-gestützter ETL-Prozesse auf konzeptioneller oder architektonischer Ebene, ohne dieses Potenzial unter kontrollierten Bedingungen empirisch zu überprüfen [9, 36]. Zweitens stützt sich ein Großteil der bestehenden Arbeiten auf klassische Verfahren des maschinellen Lernens, während der gezielte Einsatz aktueller LLMs für typische ETL-Teilaufgaben und insbesondere dessen systematischer Vergleich mit etablierten Verfahren bislang nur unzureichend untersucht ist [3, 35]. Drittens fehlt es an Untersuchungen, die mehrere aktuelle Modelle und unterschiedliche Prompting-Strategien gemeinsam und unter einheitlichen Bedingungen gegen eine klassische, regelbasierte Pipeline stellen, obwohl die Literatur sowohl die Heterogenität der Modelle als auch den starken Einfluss der Prompt-Gestaltung deutlich herausstellt [20, 23]. Viertens wird die Reproduzierbarkeit der Ergebnisse, die sich unmittelbar aus der probabilistis-

chen Natur der Modelle ergibt, in den vorhandenen Arbeiten kaum als eigenständige Untersuchungsdimension behandelt, obwohl sie für den praktischen Einsatz von zentraler Bedeutung ist [6, 7].

An genau diesen Lücken setzt die vorliegende Arbeit an. Sie verbindet die genannten Aspekte zu einer experimentellen Evaluierung, die mehrere aktuelle LLMs und unterschiedliche Prompting-Strategien anhand eines kontrolliert erstellten synthetischen Datensatzes für die Teilaufgaben Datenbereinigung, Datentransformation und Duplikaterkennung untersucht und die Ergebnisse mit einer klassisch implementierten ETL-Pipeline vergleicht. Durch die mehrfache Ausführung identischer Aufgaben wird die Reproduzierbarkeit als eigenständige Dimension einbezogen. Damit grenzt sich die Arbeit von rein konzeptionellen Beiträgen ab und liefert eine empirisch fundierte Grundlage, um die in Kapitel 1 formulierten Forschungsfragen zu beantworten und Handlungsempfehlungen für den praktischen Einsatz abzuleiten [9, 35].

4. Konzeption der experimentellen Evaluierung

Aufbauend auf den theoretischen Grundlagen aus Kapitel 2 und der in Kapitel 3 präzisierten Forschungslücke beschreibt dieses Kapitel die Konzeption der experimentellen Evaluierung. Es überführt die in Abschnitt 1.2 formulierten Forschungsfragen in ein konkretes Untersuchungsdesign und legt damit die methodische Grundlage für die folgende Umsetzung. Zunächst werden das übergeordnete Forschungsdesign und die daraus abgeleiteten Hypothesen dargelegt (Abschnitt 4.1). Anschließend werden die Auswahl der untersuchten LLMs (Abschnitt 4.2), die Definition der betrachteten ETL-Aufgaben (Abschnitt 4.3) sowie die verglichenen Prompting-Strategien (Abschnitt 4.4) beschrieben. Darauf folgt die Operationalisierung der Evaluationskriterien (Abschnitt 4.5), bevor abschließend die klassische ETL-Pipeline als Vergleichsbasis definiert wird (Abschnitt 4.6). Die konkrete technische Umsetzung des hier entworfenen Designs ist Gegenstand der Kapitel 5 bis 7.

4.1 Forschungsdesign und Hypothesen

Zur Beantwortung der Forschungsfragen wird ein experimentelles Untersuchungsdesign gewählt, das den Einsatz von LLMs für ausgewählte ETL-Teilaufgaben unter kon-

trollierten Bedingungen mit einer klassischen, regelbasierten Pipeline vergleicht. Ein experimenteller Zugang ist hier angemessen, da die Forschungslücke gerade im Fehlen belastbarer, vergleichender Evaluierungen unter einheitlichen Bedingungen besteht, während das grundsätzliche Potenzial KI-gestützter ETL-Prozesse in der Literatur bereits vielfach konzeptionell beschrieben wurde [9, 36]. Den Kern des Designs bildet ein vollständig gekreuztes Versuchsschema über die drei Faktoren Modell, Prompting-Strategie und ETL-Aufgabe: Für jede Kombination dieser Faktoren wird die Verarbeitung durchgeführt und anhand einheitlicher, in Abschnitt 4.5 definierter Kriterien bewertet, sodass sich die Einflüsse der einzelnen Faktoren systematisch isolieren lassen [6, 10].

Eine zentrale Gestaltungsentscheidung betrifft die Art und Weise, wie die Modelle in den Verarbeitungsprozess eingebunden werden. Als primärer Ausführungsmodus werden die Modelle angewiesen, einen ausführbaren ETL-Verarbeitungsschritt in Form von Python-Code zu erzeugen, der anschließend deterministisch auf den Daten ausgeführt wird, statt die Eingabedaten unmittelbar zu transformieren. Dieser Ansatz der Code-Generierung hat mehrere Vorteile: Er entkoppelt die Verarbeitung von der Datenmenge und umgeht damit die Begrenzung durch das Kontextfenster, er erzeugt eine nachvollziehbare und wiederausführbare Artefaktbasis und er stellt zugleich die unmittelbare Vergleichbarkeit mit der regelbasierten Pipeline her, die ebenfalls als programmatische Lösung auf Basis der Bibliothek Pandas umgesetzt wird [1, 4]. Der erzeugte Code wird ausgeführt, sein Ergebnis als tabellarische Ausgabedatei gespeichert und gegen eine bekannte Soll-Lösung (Ground Truth) geprüft.

Ergänzend zu diesem Vorgehen wird ein zweiter Ausführungsmodus untersucht, die direkte In-Context-Verarbeitung, bei der dem Modell die Eingabedaten unmittelbar im Prompt bereitgestellt werden und es die transformierte Ergebnistabelle ohne den Umweg über generierten Code direkt zurückgibt. Dieser Modus zielt gezielt auf die semantisch geprägten Aufgaben, bei denen ein regelbasiertes Vorgehen – ob als manuell erstellte Pipeline oder als generierter Code – an seine Grenzen stößt, weil sich das erforderliche Wissen nicht vollständig in feste Regeln überführen lässt, etwa bei der Vereinheitlichung uneinheitlich geschriebener Länderangaben oder der Erkennung unscharfer Duplikate [7, 25]. Indem das Modell sein Sprach- und Weltwissen unmittelbar auf jeden einzelnen Wert anwendet, anstatt eine verallgemeinernde Verarbeitungsvorschrift zu formulieren, lässt sich prüfen, ob die direkte Verarbeitung bei solchen Aufgaben Vorteile gegenüber der Code-Generierung erzielt. Da die Daten hierbei vollständig im Prompt mitgeführt werden, ist dieser Modus durch das Kontext- und Ausgabefenster

der Modelle begrenzt und eignet sich vor allem für Datenmengen überschaubaren Umfangs; er wird daher nicht als vollständig gekreuzter Faktor, sondern als ergänzender Vergleich auf denselben Aufgaben und Modellen geführt [6, 24].

Da LLMs aufgrund ihrer probabilistischen Natur auch bei identischer Eingabe voneinander abweichende Ausgaben erzeugen können, wird die Reproduzierbarkeit als eigenständige Untersuchungsdimension in das Design aufgenommen [6, 7]. Hierzu wird jede Konfiguration aus Modell, Prompting-Strategie und Aufgabe mehrfach ausgeführt, sodass die Streuung der Ergebnisse über die Wiederholungen hinweg quantifiziert werden kann. Die mehrfache Ausführung bezieht sich dabei auf den Generierungsschritt: Für jede Wiederholung wird das Modell erneut zur Code-Erzeugung aufgefordert, sodass sich die probabilistische Variabilität der Modelle in voneinander abweichenden Code-Artefakten niederschlagen kann, während die anschließende Ausführung eines einzelnen erzeugten Skripts selbst deterministisch bleibt. Steuerbare Einflussgrößen wie die in Abschnitt 2.5.2 eingeführte Temperatur werden über alle Durchläufe konstant und auf einen niedrigen Wert festgelegt, um die nicht durch das Modell selbst bedingte Variabilität zu minimieren und vergleichbare Bedingungen zu schaffen [6, 23]. Als unveränderliche Grundlage aller Experimente dient ein kontrolliert erzeugter synthetischer Datensatz, dessen gezielte Konstruktion in Kapitel 5 beschrieben wird und der es erlaubt, die Eigenschaften der Eingangsdaten exakt zu kennen.

Aus den Forschungsfragen und dem aufgearbeiteten Forschungsstand lassen sich die folgenden Hypothesen ableiten, die im Rahmen der Auswertung in Kapitel 8 geprüft werden. Sie konkretisieren die beiden Forschungsfragen sowie die querschnittliche Reproduzierbarkeitsdimension zu überprüfbareren Aussagen, deren Prüfung damit unmittelbar zur Beantwortung der Forschungsfragen beiträgt:

H1: Die Eignung von LLMs hängt stark vom Aufgabentyp ab. Bei klar formalisierbaren Aufgaben erreichen sie eine mit der regelbasierten Pipeline vergleichbare Ergebnisqualität, während sie bei Aufgaben mit hoher semantischer Anforderung Vorteile zeigen, mit zunehmender struktureller Komplexität jedoch an Zuverlässigkeit verlieren [7, 9].

H2: Aufwändigere Prompting-Strategien, die zusätzlichen Kontext, Beispiele oder Zwischenschritte bereitstellen, verbessern die Ergebnisqualität gegenüber einem einfachen Zero-Shot-Prompt, wobei der Nutzenzuwachs mit steigendem Aufwand

abnimmt [22, 23].

H3: Trotz niedrig gewählter Temperatur weichen wiederholte Ausführungen identischer Konfigurationen messbar voneinander ab, und die Stabilität der Ergebnisse nimmt mit zunehmender Komplexität der Aufgabe ab [6, 7].

H4: Es existiert kein über alle Aufgaben hinweg überlegener Ansatz; die Vorteilhaftigkeit eines KI-gestützten, hybriden oder klassischen Vorgehens ist vom Zusammenspiel aus Aufgabentyp, Qualitätsanforderung und Effizienzrahmen abhängig [9, 32].

4.2 Auswahl der Large Language Models

Die Auswahl der untersuchten Modelle folgt dem Ziel, ein für den praktischen Einsatz repräsentatives und zugleich aussagekräftiges Spektrum aktueller LLMs abzudecken. Wie in Abschnitt 2.4.2 dargestellt, unterscheiden sich die verfügbaren Modellfamilien nicht nur in Architektur und Parameterzahl, sondern auch in praxisrelevanten Eigenschaften wie Kontextfenster, Verarbeitungsgeschwindigkeit, Nutzungskosten und Verfügbarkeit als offenes oder geschlossenes Modell [6, 20]. Gerade diese Heterogenität begründet den Vergleich mehrerer Modelle, da sich die Eignung für ETL-Aufgaben zwischen ihnen unterscheiden kann und ein Einzelmodell keine verallgemeinerbaren Aussagen zuließe [20, 21].

Als Auswahlkriterien werden die Aktualität und Verbreitung der Modelle, die Abdeckung unterschiedlicher Anbieter sowie die Berücksichtigung sowohl geschlossener als auch offen verfügbarer Modelle herangezogen. Auf dieser Grundlage werden je ein Vertreter der etablierten geschlossenen Modellfamilien von OpenAI, Anthropic und Google sowie ein offen verfügbares, lokal ausführbares Modell aus der Llama-Reihe untersucht. Damit deckt die Auswahl unterschiedliche Kosten-, Leistungs- und Betriebsmodelle ab und erlaubt insbesondere den praktisch bedeutsamen Vergleich zwischen kommerziell betriebenen und lokal betreibbaren Modellen [6, 20]. Um die Modelle technisch austauschbar ansprechen zu können, werden sie hinter einer einheitlichen Schnittstelle gekapselt, sodass der Versuchsablauf unabhängig vom konkreten Anbieter erfolgt; die Details dieser Abstraktion werden in Kapitel 6 beschrieben. Da sich das Feld der Modelle mit hoher Geschwindigkeit weiterentwickelt, wird die exakte Versionskennung jedes Modells bei jeder Ausführung protokolliert, um die spätere Nachvollziehbarkeit und Reproduzierbarkeit der Ergebnisse sicherzustellen [6, 20].

4.3 Definition der ETL-Aufgaben

Die untersuchten ETL-Aufgaben orientieren sich an den in Abschnitt 2.2.2 beschriebenen, in der Praxis am häufigsten auftretenden Datenproblemen und decken die drei in dieser Arbeit fokussierten Teilbereiche der Transformationsphase ab: die Datenbereinigung, die Datentransformation und die Duplikaterkennung. Um nicht nur die grundsätzliche Eignung, sondern auch deren Grenzen in Abhängigkeit von der Aufgabenkomplexität zu erfassen, wird jeder Teilbereich in drei aufeinander aufbauenden Schwierigkeitsstufen operationalisiert. Diese reichen von einer einfachen, klar formalisierbaren Aufgabe über eine mittlere Stufe bis hin zu einer anspruchsvollen Variante, die semantisches Verständnis oder mehrstufige Verarbeitung erfordert. Daraus ergibt sich eine Matrix aus neun Aufgaben, die zugleich die Schwierigkeitsachse als systematische Vergleichsdimension einführt und so eine differenzierte Beurteilung der Modelle erlaubt [12, 13]. Alle Aufgaben beziehen sich auf den synthetischen Datensatz aus Kapitel 5, der ein kleines E-Commerce-Szenario mit den Tabellen Kunden, Produkte und Bestellungen abbildet, und sind so gestaltet, dass für jede von ihnen eine eindeutige Soll-Lösung existiert.

4.3.1 Datenbereinigung

Die Aufgaben zur Datenbereinigung adressieren die Beseitigung von Mängeln, die unmittelbar die Genauigkeit, Vollständigkeit und Konsistenz des Datenbestands beeinträchtigen [11, 13]. Die einfache Stufe umfasst grundlegende Bereinigungsoperationen wie das Entfernen überflüssiger Leerzeichen in Textspalten und das einheitliche Kennzeichnen fehlender Werte; diese Aufgabe ist mit wenigen Regeln vollständig beschreibbar und dient als Referenzpunkt für die grundsätzliche Verarbeitungsfähigkeit der Modelle. Die mittlere Stufe verlangt die Vereinheitlichung uneinheitlich formatierter Datumsangaben, die in mehreren unterschiedlichen Darstellungen vorliegen, auf ein einheitliches Zielformat und prüft damit die Fähigkeit zur formatübergreifenden Normalisierung. Die anspruchsvolle Stufe schließlich erfordert die semantische Vereinheitlichung von Länderangaben, die in verschiedenen Sprachen, Abkürzungen und Schreibweisen einschließlich Tippfehlern vorliegen, auf einen standardisierten Code. Gerade diese Aufgabe stellt eine für LLMs besonders aufschlussreiche Achse dar, da sie über den rein syntaktischen Abgleich hinausgeht und ein inhaltliches Verständnis der Werte voraussetzt, das mit starren Regeln nur schwer abzubilden ist [7, 25].

4.3.2 Datentransformation

Die Aufgaben zur Datentransformation betreffen die Überführung der Daten in die geforderte Struktur und Typisierung sowie die Ableitung neuer Informationen aus vorhandenen Werten. Die einfache Stufe umfasst Typkonvertierungen, etwa die Umwandlung uneinheitlich formatierter Preisangaben in einen numerischen Wert und textueller Wahrheitswerte in einen Boolean-Datentyp, und prüft damit die korrekte Behandlung von Datentypen. Die mittlere Stufe verlangt die Berechnung abgeleiteter Spalten aus bestehenden Feldern, beispielsweise die Ermittlung eines Gesamtbetrags sowie die Extraktion einzelner Datumsbestandteile, und adressiert damit zeilenweise Berechnungslogik. Die anspruchsvolle Stufe bildet die komplexeste Aufgabe der gesamten Untersuchung und verbindet mehrere Verarbeitungsschritte: Sie erfordert die Verknüpfung der drei Tabellen über ihre Schlüsselbeziehungen, eine begleitende Bereinigung der beteiligten Felder sowie eine anschließende Gruppierung und Aggregation der Ergebnisse. Damit prüft sie, ob die Modelle eine mehrstufige, aus Join, Bereinigung und Aggregation zusammengesetzte Verarbeitungslogik korrekt umsetzen können, wie sie für reale ETL-Strecken charakteristisch ist [1, 2].

4.3.3 Duplikaterkennung

Die Aufgaben zur Duplikaterkennung zielen auf die Eindeutigkeit des Datenbestands und damit auf die Identifikation und Auflösung mehrfach vorhandener Datensätze [13, 14]. Die einfache Stufe beschränkt sich auf exakte Duplikate, bei denen vollständig identische Datensätze zu entfernen sind, und ist mit einem deterministischen Abgleich vollständig lösbar. Die mittlere Stufe verlangt die schlüsselbasierte Entdopplung anhand eines eindeutigen Merkmals, wobei bei mehreren übereinstimmenden Datensätzen anhand einer definierten Regel der zu behaltende Datensatz ausgewählt werden muss. Die anspruchsvolle Stufe adressiert schließlich unscharfe Duplikate (Fuzzy-Duplikate), bei denen derselbe reale Sachverhalt in abweichenden Schreibweisen vorliegt, etwa durch Tippfehler, unterschiedliche Groß- und Kleinschreibung oder geringfügig abweichende Formatierungen. Da hier ein exakter Abgleich nicht ausreicht und stattdessen eine Ähnlichkeitsbetrachtung erforderlich ist, bildet diese Aufgabe die zweite zentrale Achse, an der sich die semantischen Fähigkeiten der Modelle gegenüber regelbasierten Verfahren zeigen [7, 13].

4.4 Prompting-Strategien

Da LLMs ihre Aufgaben primär über das In-Context-Lernen lösen, kommt der Gestaltung der Eingabe eine entscheidende Bedeutung für die Ergebnisqualität zu, weshalb die Prompting-Strategie als eigenständiger Untersuchungsfaktor in das Design aufgenommen wird [22, 23]. Angesichts der in Abschnitt 2.5.1 angesprochenen uneinheitlichen Begriffsbildung im Feld werden für diese Arbeit klar abgegrenzte Strategien definiert, die sich in ihrem Aufwand und dem Umfang der bereitgestellten Hilfestellung systematisch unterscheiden und gegeneinander verglichen werden.

Den Ausgangspunkt bildet das Zero-Shot-Prompting, bei dem das Modell die Aufgabe allein anhand einer präzisen Anweisung und ohne weitere Beispiele bearbeitet. Diese Strategie dient als Referenzbasis, die misst, was ein Modell ohne zusätzliche Hilfestellung aus einer reinen Aufgabenbeschreibung erzeugt [19, 22]. Darauf aufbauend wird das Few-Shot-Prompting untersucht, das der Anweisung einige beispielhafte Ein- und Ausgabepaare beifügt, anhand derer das Modell das gewünschte Verhalten ableitet und das die Ergebnisqualität bei vielen Aufgaben spürbar verbessert [19, 23]. Als dritte Strategie wird das Chain-of-Thought-Prompting einbezogen, bei dem das Modell zur Entwicklung seiner Lösung in nachvollziehbaren Zwischenschritten angeleitet wird, was sich insbesondere bei den mehrstufigen Aufgaben dieser Untersuchung als wirksam erweisen könnte [22, 23]. Ergänzend wird eine schema-angereicherte Variante betrachtet, die der Anweisung eine explizite Beschreibung des erwarteten Zielschemas beifügt, um die strukturelle Korrektheit der Ausgabe zu unterstützen. Um faire Vergleichsbedingungen zu gewährleisten, werden die Strategien für alle Aufgaben nach einem einheitlichen Aufbau formuliert und versioniert vorgehalten, sodass jede eingesetzte Prompt-Variante eindeutig dokumentiert und wiederverwendbar bleibt [23, 24].

4.5 Evaluationskriterien

Damit die Ergebnisse aller Verfahren über Modelle, Prompting-Strategien und Aufgaben hinweg vergleichbar gemessen werden können, werden die Bewertungskriterien aus den in Abschnitt 2.2.1 eingeführten Dimensionen der Datenqualität abgeleitet und operationalisiert [10, 12]. Grundlage jeder Bewertung ist der Abgleich der erzeugten Ausgabedatei mit der für jede Aufgabe bekannten Soll-Lösung. Die folgenden Unterabschnitte beschreiben die vier herangezogenen Kriterien Genauigkeit, Vollständigkeit, Konsistenz und Effizienz. Die Stabilität der Ergebnisse – als querschnittliche Repro-

duzierbarkeitsdimension über beide Forschungsfragen hinweg – wird nicht als separates Einzelkriterium erhoben, sondern daraus abgeleitet, wie stark die genannten Qualitätskennzahlen über die wiederholten Ausführungen einer Konfiguration streuen: Eine geringe Streuung entspricht dabei einer hohen, eine große Streuung einer geringen Stabilität [6, 7].

4.5.1 Genauigkeit

Die Genauigkeit erfasst, inwieweit die erzeugten Werte mit der Soll-Lösung übereinstimmen, und bildet das zentrale Korrektheitskriterium [11, 12]. Sie wird als Anteil der korrekt verarbeiteten Werte am Gesamtbestand operationalisiert, sodass auch teilweise korrekte Ergebnisse differenziert bewertet werden können und nicht jede Abweichung gleichermaßen ins Gewicht fällt. Voraussetzung für diesen Vergleich ist die Übereinstimmung der grundlegenden Struktur, also der Spaltenmenge und der Datensatzanzahl; weicht diese ab, wird dies gesondert ausgewiesen, da in einem solchen Fall kein sinnvoller Abgleich möglich ist. Auf diese Weise lässt sich die Genauigkeit als kontinuierliche Kennzahl zwischen vollständiger Übereinstimmung und vollständiger Abweichung abbilden.

4.5.2 Vollständigkeit

Die Vollständigkeit gibt an, inwieweit das Ergebnis alle geforderten Datensätze und Felder enthält, ohne dass erwartete Werte fehlen oder unzulässig entfernt werden [11, 12]. Sie wird über den Abgleich der erzeugten Struktur mit der Soll-Lösung erfasst, insbesondere über das Vorhandensein aller erwarteten Spalten und die korrekte Anzahl der Datensätze. Dieses Kriterium ist gerade für die Aufgaben der Duplikaterkennung von Bedeutung, bei denen sowohl ein unzureichendes als auch ein übermäßiges Entfernen von Datensätzen die Vollständigkeit verletzt, und es ergänzt die Genauigkeit um die Frage, ob der Umfang des Ergebnisses dem erwarteten Umfang entspricht.

4.5.3 Konsistenz

Die Konsistenz bezieht sich auf die Widerspruchsfreiheit und Einheitlichkeit des Ergebnisses, insbesondere auf die Übereinstimmung der Datentypen und Formate mit dem erwarteten Zielschema [11, 12]. Bewertet wird, ob die Ausgabe das geforderte Schema einhält, ob die einzelnen Felder die erwarteten Datentypen aufweisen und ob Werte

einheitlich dargestellt werden. Dieses Kriterium ist vor allem für die Transformation-saufgaben relevant, bei denen die korrekte Typisierung und Formatierung ein wesentliches Ergebnismerkmal darstellt, und es prüft die strukturelle Verlässlichkeit der erzeugten Daten unabhängig von der inhaltlichen Korrektheit einzelner Werte.

4.5.4 Effizienz

Die Effizienz erfasst den für die Verarbeitung erforderlichen Aufwand und ergänzt die qualitätsbezogenen Kriterien um die für den praktischen Einsatz entscheidende Wirtschaftlichkeitsdimension [20, 24]. Erhoben werden die Verarbeitungszeit eines Aufrufs, der Verbrauch an Token sowie die daraus resultierenden Nutzungskosten. Diese Kennzahlen sind insbesondere für die vergleichende Bewertung der Prompting-Strategien relevant, da aufwändigere Strategien mit längeren Eingaben einen höheren Token-Verbrauch verursachen und somit zwischen Ergebnisqualität und Effizienz abzuwägen ist [23, 24]. Zugleich ermöglichen sie den praxisnahen Vergleich zwischen kommerziell betriebenen und lokal ausgeführten Modellen sowie die Gegenüberstellung mit der klassischen Pipeline, deren Ausführung keine vergleichbaren Modellkosten verursacht.

4.6 Vergleich gegen eine manuelle ETL-Pipeline

Um die Ergebnisse der LLM-gestützten Verarbeitung einordnen zu können, wird als Vergleichsmaßstab eine klassische, regelbasiert implementierte ETL-Pipeline herangezogen, die dieselben neun Aufgaben auf demselben Datensatz löst. Diese Pipeline bildet den etablierten Stand der Technik ab, bei dem die Verarbeitungslogik manuell und explizit in Form von Regeln und Skripten vorgegeben wird, und sie wird ebenfalls programmatisch auf Basis der Bibliothek Pandas umgesetzt, um eine technisch vergleichbare Ausgangslage herzustellen [3, 10]. Sie dient als Referenzpunkt, gegen den sich die Stärken und Schwächen der KI-gestützten Ansätze messen lassen, und liefert damit die Grundlage zur Beantwortung von FF1.

Charakteristisch für diese Vergleichsbasis ist, dass sie deterministische und vollständig nachvollziehbare Ergebnisse erzeugt: Identische Eingaben führen stets zu identischen Ausgaben, und die zugrunde liegende Logik ist explizit einsehbar [3, 4]. Damit bildet die regelbasierte Pipeline insbesondere bei den klar formalisierbaren Aufgaben die Referenz, während gerade bei den semantisch geprägten Aufgaben der Länder-Vereinheitlichung und der Fuzzy-Duplikaterkennung erwartet wird, dass sich die Grenzen rein regelbasierter Verfahren zeigen. Diese arbeiten zwar auch bei solchen Auf-

gaben deterministisch, erfassen die zugrunde liegende semantische Ähnlichkeit jedoch nur eingeschränkt. Die Pipeline wird mit den identischen Evaluationskriterien aus Abschnitt 4.5 bewertet, sodass ein unmittelbarer Vergleich zwischen klassischem und KI-gestütztem Vorgehen unter gleichen Bedingungen möglich ist und sich auf dieser Grundlage in Kapitel 9 Handlungsempfehlungen ableiten lassen. Ein hybrider Ansatz, der beide Verfahren kombiniert, ist dabei nicht selbst Gegenstand der Evaluierung; da sich die Nachvollziehbarkeit regelbasierter und die Flexibilität KI-gestützter Verfahren in der Praxis jedoch häufig ergänzen, wird er als naheliegende Konsequenz des Vergleichs in den Handlungsempfehlungen aufgegriffen [9, 32].

5. Erstellung des synthetischen Datensatzes

Das in Kapitel 4 entworfene Untersuchungsdesign setzt einen Datensatz voraus, dessen Eigenschaften exakt bekannt sind: Nur wenn die enthaltenen Datenqualitätsprobleme gezielt eingebracht wurden und für jede Aufgabe eine eindeutige Soll-Lösung existiert, lassen sich die in Abschnitt 4.5 definierten Kriterien überhaupt objektiv messen. Dieses Kapitel dokumentiert daher die Erstellung des synthetischen Datensatzes, der in Abschnitt 4.1 bereits als unveränderliche Grundlage aller Experimente eingeführt wurde. Es schließt unmittelbar an die in Abschnitt 4.3 definierten neun ETL-Aufgaben an und zeigt, wie das kleine E-Commerce-Szenario mit den Tabellen Kunden, Produkte und Bestellungen so konstruiert wird, dass genau die dort geforderten Probleme in kontrolliertem Umfang vorliegen. Zunächst werden die aus dem Forschungsdesign abgeleiteten Anforderungen begründet (Abschnitt 5.1), anschließend das Datenmodell und die saubere Zielstruktur beschrieben (Abschnitt 5.2). Darauf folgt die detaillierte Zuordnung der gezielt eingebauten Qualitätsprobleme zu den einzelnen Aufgaben (Abschnitt 5.3), bevor das zweistufige Generierungsverfahren (Abschnitt 5.4) und die abschließende Validierung des Datensatzes (Abschnitt 5.5) dargelegt werden. Der so erzeugte Datensatz bildet die Eingabe für die technische Umsetzung in Kapitel 6 und die Durchführung der Experimente in Kapitel 7.

5.1 Anforderungen an den Datensatz

Die Anforderungen an den Datensatz ergeben sich unmittelbar aus den Forschungsfragen und dem in Kapitel 4 festgelegten Design. Eine erste Anforderung betrifft

die Repräsentativität: Der Datensatz muss die in Abschnitt 2.2.2 beschriebenen, in der Praxis typischen Datenqualitätsprobleme abbilden, damit die Ergebnisse über das konkrete Szenario hinaus aussagekräftig sind und die Aufgaben nicht künstlich vereinfacht wirken [12, 13]. Zugleich soll er nicht beliebige, sondern gerade jene Probleme enthalten, die den neun in Abschnitt 4.3 definierten Aufgaben entsprechen, sodass jede Aufgabe an einem tatsächlich vorhandenen Mangel ansetzt.

Eine zweite, für das gewählte experimentelle Vorgehen zentrale Anforderung ist die vollständige Kontrollierbarkeit. Da die Qualität der Verarbeitung durch den Abgleich mit einer bekannten Soll-Lösung gemessen wird, müssen sowohl der fehlerfreie Sollzustand als auch Art und Umfang der eingebrachten Fehler exakt bekannt sein. Bei realen Datenbeständen wäre diese eindeutige Referenz nur mit erheblichem manuellem Annotationsaufwand und verbleibender Unsicherheit herstellbar, weshalb die Literatur den Einsatz synthetischer Daten gerade dann empfiehlt, wenn eine kontrollierte und exakt bekannte Ausgangslage benötigt wird [29, 38]. Die in Abschnitt 3.2 referierten Vorbehalte gegenüber synthetischen Daten betreffen primär deren Einsatz als Trainingsmaterial; für die hier verfolgte Rolle als kontrollierte Testgrundlage überwiegt hingegen der Vorteil der bekannten Ground Truth [27, 31].

Eine dritte Anforderung ist die Reproduzierbarkeit des Datensatzes selbst. Da die Reproduzierbarkeit der Modellergebnisse eine eigenständige, querschnittliche Untersuchungsdimension bildet, darf die Datengrundlage keine zusätzliche Variabilität einführen; die Generierung muss deterministisch sein, sodass sie jederzeit identisch wiederholt werden kann [6, 7]. Schließlich soll der Datensatz eine vierte Anforderung erfüllen, die zwischen Vielfalt und Handhabbarkeit vermittelt: Er muss groß und vielfältig genug sein, um die drei Schwierigkeitsstufen jeder Aufgabe sinnvoll auszuprägen, zugleich aber überschaubar genug bleiben, um die in Abschnitt 4.5.4 erhobenen Aufwandskennzahlen nicht durch reine Datenmenge zu dominieren und die wiederholte Ausführung aller Konfigurationen praktikabel zu halten.

5.2 Datenmodell und -struktur

Als Anwendungsszenario dient ein bewusst einfach gehaltenes E-Commerce-Datenmodell, das aus den drei Tabellen Kunden, Produkte und Bestellungen besteht. Diese Wahl ist kein Selbstzweck, sondern folgt aus den Aufgaben: Das Szenario ist allgemein verständlich, deckt sämtliche in Abschnitt 4.3 geforderten Problemtypen ab und stellt über die Beziehung der Bestellungen zu Kunden und Produkten zugleich die Schlüs-

selstruktur bereit, die für die anspruchsvolle Transformationsaufgabe aus Join, Bereinigung und Aggregation benötigt wird [1, 2]. Die Tabelle der Bestellungen verweist über Fremdschlüssel auf Kunden und Produkte und verbindet die drei Tabellen so zu einem kleinen relationalen Modell.

Grundlage des Modells ist die Definition eines sauberen Zielschemas für jede Tabelle, das den fehlerfreien Sollzustand festlegt. Für die Kundentabelle umfasst dieses Schema eine eindeutige Kundennummer, den vollständigen Namen, die E-Mail-Adresse, einen zweistelligen Länder-Code nach ISO-3166-1-alpha-2 sowie das Registrierungsdatum. Die Produkttabelle besteht aus einer Produktnummer, der Bezeichnung, einer Kategorie, dem Preis als nichtnegativem Dezimalwert und einem Wahrheitswert zur Verfügbarkeit. Die Bestelltabelle schließlich verknüpft Bestell-, Kunden- und Produktnummer mit der bestellten Menge, dem Bestelldatum und dem Gesamtbetrag. Dieses saubere Schema erfüllt eine doppelte Funktion: Es beschreibt einerseits das von den Modellen zu erreichende Zielformat und dient andererseits, in seiner konkret erzeugten Ausprägung, als bekannte Referenz für die Bewertung. Die typisierte, maschinenlesbare Festlegung des Schemas stellt dabei sicher, dass die als Referenz erzeugten Daten strikt schemakonform sind und damit als verlässliche Ground Truth taugen [11, 12].

Der Umfang des Datensatzes ist so gewählt, dass er die genannte Abwägung zwischen Vielfalt und Handhabbarkeit einlöst. Die Kundentabelle umfasst in ihrer sauberen Fassung fünfhundert Datensätze, die Produkttabelle einhundert und die Bestelltabelle zweitausend Datensätze. Diese Größenordnung ist einerseits ausreichend, um Datenqualitätsprobleme in statistisch merklichem, aber nicht triviales Ausmaß einzustreuen und die Aggregation über Länder und Kategorien mit belastbaren Häufigkeiten zu füllen, andererseits klein genug, um die in Abschnitt 4.6 vorgesehene wiederholte Ausführung über alle Kombinationen aus Modell, Prompting-Strategie und Aufgabe ohne unverhältnismäßigen Zeit- und Kostenaufwand zu ermöglichen.

5.3 Eingebaute Datenqualitätsprobleme

Kern der Datensatzkonstruktion ist das gezielte Einbringen von Qualitätsproblemen, das dieser Abschnitt entlang der drei in Abschnitt 4.3 definierten Aufgabenkategorien beschreibt. Jedes Problem ist dabei nicht beliebig gewählt, sondern korrespondiert mit genau einer Aufgabe und einer der in Abschnitt 2.2.1 eingeführten Qualitätsdimensionen, sodass sich die spätere Bewertung lückenlos auf einen bewusst gesetzten Mangel zurückführen lässt [12, 13]. Auf diese Weise ist die in Abschnitt 4.3 eingeführte Ma-

trix aus neun Aufgaben und drei Schwierigkeitsstufen unmittelbar in der Struktur des Datensatzes verankert.

Die Probleme der *Datenbereinigung* sind ausschließlich in der Kundentabelle verortet und in aufsteigender Schwierigkeit gestaffelt. Für die einfache Stufe werden in einem Teil der Namensfelder zusätzliche Leerzeichen vorangestellt und angehängt und in einem Teil der Länderangaben die Werte gänzlich entfernt, wodurch die für diese Stufe geforderte Beseitigung überflüssiger Leerzeichen und die Kennzeichnung fehlender Werte adressiert werden [11, 13]. Die mittlere Stufe entsteht dadurch, dass das Registrierungsdatum in vier unterschiedlichen Darstellungen abgelegt wird, nämlich als ISO-Datum, im deutschen Format, im US-amerikanischen Format mit ausgeschriebenem Monat sowie als Unix-Zeitstempel; die Aufgabe der formatübergreifenden Normalisierung besitzt damit im Datensatz eine konkrete Entsprechung. Die anspruchsvolle Stufe schließlich wird durch die Verfälschung der Länderangaben erzeugt, indem der saubere ISO-Code durch wechselnde Schreibweisen, Sprachen, Abkürzungen und gezielte Tippfehler ersetzt wird, etwa "Deutschland", "Germany" oder "Deutshcland" anstelle von "DE". Gerade diese Verfälschung bildet die in Abschnitt 4.3.1 hervorgehobene semantische Achse ab, da ihre Auflösung ein inhaltliches Verständnis der Werte und nicht nur einen syntaktischen Abgleich erfordert [7, 25].

Die Probleme der *Datentransformation* betreffen die Produkt- und die Bestelltabelle und knüpfen an die Dimension der Konsistenz an. In der Produkttabelle werden die Preisangaben von ihrem numerischen Sollwert in uneinheitliche textuelle Schreibweisen überführt, etwa mit Dezimalkomma, vorangestelltem Euro-Zeichen oder nachgestelltem Währungskürzel, und die Verfügbarkeit wird von einem Wahrheitswert in wechselnde textuelle Entsprechungen umgeschrieben. Damit besitzt die einfache Transformationsaufgabe der Typkonvertierung eine unmittelbare Grundlage. Die mittlere Stufe erfordert keine eigene Verfälschung, sondern nutzt die vorhandene Struktur der Bestelltabelle, aus der sich der Gesamtbetrag sowie einzelne Datumsbestandteile ableiten lassen, und prüft so die zeilenweise Berechnungslogik. Die anspruchsvolle Stufe schließlich ergibt sich aus dem Zusammenspiel aller drei Tabellen: Sie verlangt deren Verknüpfung über die Schlüsselbeziehungen, eine begleitende Bereinigung der zuvor beschriebenen Mängel und eine abschließende Aggregation über Land und Produktkategorie. Diese Aufgabe bündelt damit bewusst mehrere der eingebauten Probleme in einem einzigen, mehrstufigen Verarbeitungsschritt, wie er für reale ETL-Strecken charakteristisch ist [1, 2].

Die Probleme der *Duplikaterkennung* zielen auf die Eindeutigkeit der Kundentabelle und werden durch das Hinzufügen zweier Arten von Doppelungen erzeugt. Für die einfache und die mittlere Stufe werden vollständig identische Datensätze eingefügt, die sowohl als exakte Duplikate zu erkennen sind als auch, über die mehrfach vergebene E-Mail-Adresse, die schlüsselbasierte Entdoppelung ermöglichen. Für die anspruchsvolle Stufe werden unscharfe Duplikate gebildet, bei denen derselbe Kunde mit geringfügig abweichendem Namen und abweichender E-Mail-Schreibweise erneut auftritt, etwa durch vertauschte Buchstaben, veränderte Groß- und Kleinschreibung, eingefügte Mittelinitialen oder zusätzliche Punkte im lokalen Teil der Adresse. Da diese Fuzzy-Duplikate nicht durch exakten Abgleich, sondern nur über eine Ähnlichkeitsbetrachtung auflösbar sind, bilden sie die in Abschnitt 4.3.3 beschriebene zweite semantische Achse der Untersuchung [7, 13]. Der Anteil der eingebrachten Probleme ist über alle Kategorien hinweg so bemessen, dass er einerseits merkbare Auswirkungen auf die Ergebniskennzahlen hat, andererseits den grundsätzlichen Charakter des Datensatzes als überwiegend sauberen Bestand nicht verfälscht; so betreffen die zusätzlichen exakten und unscharfen Duplikate jeweils rund vier Prozent der Kundendatensätze, während die fehlenden Länderangaben und die zusätzlichen Leerzeichen in vergleichbar begrenztem Umfang gesetzt werden.

5.4 Generierungsverfahren und verwendete Werkzeuge

Die Erzeugung des Datensatzes folgt einem zweistufigen Prinzip, das die in Abschnitt 5.1 geforderte Kontrollierbarkeit unmittelbar umsetzt. Im ersten Schritt werden für jede Tabelle vollständig saubere, strikt schemakonforme Daten erzeugt, die den fehlerfreien Sollzustand verkörpern. Im zweiten Schritt werden aus dieser sauberen Fassung durch eine kontrollierte Verfälschung die tatsächlich zu verarbeitenden Rohdaten abgeleitet, während die saubere Fassung selbst als Referenz erhalten bleibt. Dieses Vorgehen kehrt den üblichen Ablauf einer ETL-Strecke bewusst um: Nicht die Bereinigung wird nachträglich auf unbekannte Rohdaten angewandt, sondern die Verunreinigung wird gezielt auf bekannte saubere Daten aufgebracht, sodass die exakte Umkehrung jeder Verfälschung und damit die eindeutige Soll-Lösung jeder Aufgabe von vornherein feststeht [29, 38].

Für die Erzeugung realistisch wirkender Basiswerte wie Namen und E-Mail-Adressen wird eine etablierte Bibliothek zur Generierung synthetischer Testdaten eingesetzt, deren deutschsprachige Lokalisierung dem Szenario angemessene Werte liefert. Die kontrollierte Verfälschung, also das Einstreuen von Leerzeichen, fehlenden Werten,

uneinheitlichen Formaten, verfälschten Länderangaben sowie exakten und unscharfen Duplikaten, erfolgt anschließend durch eigens implementierte Routinen, die jede der in Abschnitt 5.3 beschriebenen Problemklassen erzeugen. Sowohl die Basisdaten als auch die Verfälschung werden über einen zentralen Zufallszahlengenerator gesteuert, der mit einem festen Startwert initialisiert wird. Dadurch ist die Generierung vollständig deterministisch: Derselbe Startwert führt stets zu einem identischen Datensatz, während unterschiedliche Startwerte voneinander unabhängige, aber strukturell gleichartige Datensätze erzeugen. Diese Determiniertheit stellt sicher, dass die untersuchte Variabilität allein den Modellen und nicht der Datengrundlage zuzurechnen ist [6, 7]. Die erzeugten Rohdaten werden in einem gängigen tabellarischen Austauschformat abgelegt und dienen zusammen mit der separat gespeicherten sauberen Referenz als Eingabe für die in Kapitel 6 beschriebene Verarbeitungsumgebung.

5.5 Validierung des Datensatzes

Bevor der Datensatz als Grundlage der Experimente verwendet wird, ist zu prüfen, ob er die in Abschnitt 5.1 formulierten Anforderungen tatsächlich erfüllt. Die Validierung setzt zunächst an der sauberen Referenz an: Da diese als maßgebliche Ground Truth dient, wird ihre strikte Konformität mit dem definierten Zielschema überprüft, sodass sichergestellt ist, dass die als korrekt geltenden Werte die geforderten Datentypen, Wertebereiche und Formate einhalten [11, 12]. Nur unter dieser Voraussetzung ist der spätere Abgleich der Modellergebnisse mit der Referenz aussagekräftig.

In einem zweiten Schritt wird geprüft, ob die eingebauten Qualitätsprobleme in der beabsichtigten Art und im vorgesehenen Umfang vorhanden sind. Kontrolliert wird insbesondere, dass jede der neun Aufgaben aus Abschnitt 4.3 auf einen tatsächlich vorliegenden Mangel trifft, dass die verschiedenen Datums- und Preisformate wie vorgesehen auftreten und dass sich die exakten und unscharfen Duplikate eindeutig auf ihre jeweiligen Ausgangsdatsätze zurückführen lassen. Damit ist zugleich gewährleistet, dass für jede Aufgabe eine eindeutige Soll-Lösung existiert, was die in Abschnitt 4.5 definierten Kriterien überhaupt erst berechenbar macht [10, 13]. Ergänzend wird die Reproduzierbarkeit der Generierung dadurch bestätigt, dass die wiederholte Ausführung mit identischem Startwert einen bitgenau identischen Datensatz liefert.

Mit dieser Validierung steht ein Datensatz zur Verfügung, dessen Sollzustand, Fehlerbild und Erzeugung vollständig bekannt und nachvollziehbar sind und der damit die kontrollierte Vergleichsgrundlage für die weitere Untersuchung bildet. Zugleich ist die

dem Vorgehen innewohnende Einschränkung zu benennen: Ein synthetischer Datensatz kann die Vielfalt und die unvorhergesehenen Unregelmäßigkeiten realer Datenbestände nur annähern, weshalb die aus ihm gewonnenen Ergebnisse unter diesem Vorbehalt zu interpretieren und im Rahmen der Limitationen in Kapitel 10 erneut aufzugreifen sind [29, 31]. Auf der so geschaffenen Grundlage kann im folgenden Kapitel 6 die technische Umsetzung der Verarbeitungsumgebung beschrieben werden, welche die hier erzeugten Daten sowohl der klassischen Pipeline als auch den LLM-gestützten Verfahren zuführt.

6. Implementierung

Das vorangegangene Kapitel 4 hat das Untersuchungsdesign entworfen und der Datensatz aus Kapitel 5 stellt die unveränderliche Datengrundlage bereit. Dieses Kapitel beschreibt, wie dieses Design in ein lauffähiges Softwaresystem überführt wurde, mit dem sich die vollständige Matrix aus Modell, Prompting-Strategie und ETL-Aufgabe automatisiert durchführen und einheitlich bewerten lässt. Zunächst wird die modulare Systemarchitektur und die Versuchsumgebung dargestellt (Abschnitt 6.1), gefolgt von den eingesetzten Technologien und Bibliotheken (Abschnitt 6.2). Anschließend werden die Umsetzung der klassischen Vergleichspipeline (Abschnitt 6.3), die Integration der LLMs über eine einheitliche Schnittstelle (Abschnitt 6.4) sowie die konkrete Realisierung der Prompting-Strategien (Abschnitt 6.5) beschrieben. Den Abschluss bildet das automatisierte Mess- und Auswertungsverfahren (Abschnitt 6.6), das die in Abschnitt 4.5 definierten Kriterien technisch operationalisiert und damit die Grundlage für die Durchführung in Kapitel 7 schafft.

6.1 Systemarchitektur und Versuchsumgebung

Das System ist als modulares Python-Projekt aufgebaut, dessen Komponenten entlang der einzelnen Verarbeitungsschritte getrennt sind und über klar umrissene Schnittstellen zusammenwirken. Diese Trennung folgt dem Ziel, die einzelnen Faktoren des Versuchsdesigns unabhängig voneinander austauschen zu können, ohne den übrigen Ablauf zu verändern, und erleichtert zugleich die Nachvollziehbarkeit der Ergebnisse [4, 10]. Ein Datenmodul kapselt die Erzeugung des synthetischen Datensatzes, die Referenzlösungen sowie die Definition der neun Aufgaben; ein Modellmodul stellt die Anbindung der LLMs bereit; ein Modul für die klassische Pipeline enthält die regelbasierte Vergleichslösung; ein Experimentmodul steuert den Ablauf und die Ausführung

des erzeugten Codes; ein Evaluationsmodul bewertet die Ergebnisse; und ein Auswertungsmodul bereitet die erhobenen Messdaten für die Analyse auf.

Der grundlegende Datenfluss folgt für jede Konfiguration demselben Muster: Ausgehend vom synthetischen Datensatz und der Aufgabenbeschreibung wird ein Prompt erzeugt, an das jeweilige Modell übergeben und dessen Antwort weiterverarbeitet. Entsprechend der in Abschnitt 4.1 getroffenen Gestaltungsentscheidung erzeugt das Modell dabei nicht das Ergebnis unmittelbar, sondern ein ausführbares Verarbeitungsskript, das anschließend deterministisch auf den Daten ausgeführt wird. Das so entstandene Ergebnis wird als tabellarische Ausgabedatei gespeichert, gegen die bekannte Soll-Lösung geprüft und gemeinsam mit allen erhobenen Kennzahlen in einem unveränderlichen Ergebnisdatensatz protokolliert [1, 6]. Sämtliche steuerbaren Randbedingungen sind in einer zentralen Konfiguration gebündelt, die unter anderem den Zufallskern für die Datenerzeugung, die konstant niedrig gehaltene Temperatur, die Obergrenze der Ausgabe-Token sowie die Anzahl der Wiederholungen je Konfiguration festlegt. Diese zentrale und explizit hinterlegte Parametrisierung ist eine wesentliche Voraussetzung für die in Abschnitt 4.5 als querschnittliche Dimension eingeführte Reproduzierbarkeit, da sie die Bedingungen jedes Durchlaufs eindeutig dokumentiert [6, 7]. Die Ausführung erfolgt durchgängig in einer kontrollierten lokalen Umgebung.

6.2 Eingesetzte Technologien und Bibliotheken

Als Implementierungssprache dient Python, da es im Umfeld der Datenverarbeitung und des maschinellen Lernens die verbreitetste Sprache ist und mit einem umfangreichen Ökosystem einschlägiger Bibliotheken sowie offiziellen Schnittstellen aller untersuchten Modellanbieter unterstützt wird [17, 39]. Die eigentliche Datenverarbeitung stützt sich sowohl in der klassischen Pipeline als auch im vom Modell erzeugten Code auf die Bibliothek Pandas, die sich als De-facto-Standard für tabellarische Verarbeitung etabliert hat. Ihre einheitliche Verwendung stellt sicher, dass die regelbasierte und die KI-gestützte Lösung auf derselben technischen Grundlage arbeiten und damit unmittelbar vergleichbar bleiben [1, 4]. Sämtliche Zwischen- und Ergebnistabellen werden im spaltenorientierten Parquet-Format ausgetauscht, da dieses die Datentypen der Spalten mitführt und so einen typerhaltenden Abgleich zwischen erzeugter Ausgabe und Soll-Lösung ermöglicht, wie er für das Konsistenzkriterium aus Abschnitt 4.5.3 benötigt wird.

Die Anbindung der Modelle erfolgt über die jeweils offiziellen Softwarebibliotheken der

Anbieter für die geschlossenen Modelle von OpenAI, Anthropic und Google sowie über eine lokale Laufzeitumgebung für das offen verfügbare Modell der Llama-Reihe. Diese heterogenen Schnittstellen werden, wie in Abschnitt 6.4 beschrieben, hinter einer einheitlichen Abstraktion zusammengeführt, sodass der Versuchsablauf unabhängig vom konkreten Anbieter bleibt [6, 20]. Ergänzend kommen etablierte Hilfsbibliotheken zum Einsatz: eine typsichere Konfigurationsverwaltung zur Bündelung der Versuchsparameter, eine Auszeichnungssprache zur versionierten Ablage der Prompt-Vorlagen sowie Werkzeuge zur Visualisierung und interaktiven Auswertung, mit denen die in Kapitel 8 dargestellten Tabellen und Abbildungen erzeugt werden. Als repräsentative Vertreter der jeweiligen Anbieter wird je ein aktuelles Modell eingesetzt, dessen exakte Versionskennung bei jedem Aufruf protokolliert wird, um der raschen Weiterentwicklung des Modellfeldes Rechnung zu tragen und die spätere Nachvollziehbarkeit zu sichern [6, 20].

6.3 Umsetzung der klassischen ETL-Pipeline

Die in Abschnitt 4.6 als Vergleichsmaßstab definierte klassische Pipeline ist als regelbasierte Lösung umgesetzt, die dieselben neun Aufgaben ausschließlich mit fest vorgegebener Verarbeitungslogik und ohne Zugriff auf ein Sprachmodell löst. Für die klar formalisierbaren Aufgaben, etwa das Entfernen exakter Duplikate oder die Typkonvertierung, nutzt sie dieselben deterministischen Routinen, die auch der Erzeugung der Soll-Lösung zugrunde liegen; für diese Aufgaben ist die explizit formulierte Regel zugleich die korrekte Lösung, sodass hier definitionsgemäß eine hohe Ergebnisqualität zu erwarten ist [3, 4].

Von besonderer Bedeutung für die Aussagekraft des Vergleichs ist die bewusste Gestaltung der Pipeline bei den semantisch geprägten Aufgaben. Für die Vereinheitlichung der Länderangaben greift sie auf eine absichtlich begrenzte Zuordnungstabelle zurück, die zwar ausgeschriebene Ländernamen und Standardcodes abdeckt, jedoch keine Tippfehler, untypischen Abkürzungen oder umgangssprachlichen Bezeichnungen berücksichtigt; nicht auflösbare Werte werden einheitlich als unbekannt gekennzeichnet. Für die Erkennung unscharfer Duplikate bildet sie einen normalisierten Schlüssel aus Name und E-Mail-Bestandteil und entfernt darüber Mehrfachnennungen, kann damit aber nur solche Abweichungen erfassen, die sich durch einfache Normalisierung angleichen lassen. Diese Heuristiken sind realistisch für ein rein regelbasiertes Vorgehen, stoßen jedoch genau dort an ihre Grenzen, wo ein inhaltliches Verständnis der Werte erforderlich wäre. Gerade an dieser Stelle wird der in Hypothese H1 erwartete Unter-

schied zwischen regelbasierter und KI-gestützter Verarbeitung messbar [7, 13]. Da die Pipeline vollständig deterministisch arbeitet und identische Eingaben stets zu identischen Ausgaben führen, dient sie im weiteren Verlauf als stabiler und nachvollziehbarer Referenzpunkt, gegen den sich die Ergebnisse der Modelle einordnen lassen.

6.4 Integration der LLMs in die Pipeline

Um die in Abschnitt 4.2 ausgewählten Modelle trotz ihrer unterschiedlichen Schnittstellen einheitlich ansprechen zu können, werden sie hinter einer gemeinsamen Abstraktion gekapselt. Jeder Anbieter implementiert dieselbe abstrakte Schnittstelle, die einen Aufruf entgegennimmt und eine standardisierte Antwort zurückliefert, in der neben dem erzeugten Text auch die exakte Modellkennung, der Verbrauch an Eingabe- und Ausgabe-Token sowie die Antwortzeit enthalten sind. Eine vorgeschaltete Erzeugungsinstanz liefert anhand eines Anbieternamens die passende Implementierung, sodass der Versuchsablauf vollständig unabhängig vom konkreten Modell bleibt und die Faktorachse Modell ohne Eingriffe in die übrige Logik variiert werden kann [6, 20]. Diese standardisierte Antwort ist zugleich die Datenbasis für die spätere Effizienzbewertung nach Abschnitt 4.5.4, da sie die dafür benötigten Kennzahlen bei jedem Aufruf unmittelbar mitführt.

Der eigentliche Verarbeitungsschritt folgt dem Ansatz der Code-Generierung: Aus der Modellantwort wird der enthaltene Python-Code extrahiert und anschließend in einem eigenen Betriebssystemprozess mit fester Zeitbegrenzung ausgeführt. Ein Durchlauf gilt nur dann als erfolgreich, wenn der Prozess fehlerfrei endet und tatsächlich die erwartete Ausgabedatei erzeugt wurde; andernfalls wird das Ergebnis als fehlgeschlagene Ausführung protokolliert. Da an dieser Stelle von einem Modell erzeugter Code ausgeführt wird, ist die Ausführung in einen separaten Prozess mit Zeitlimit ausgelagert. Dieses Vorgehen ist für die kontrollierte, lokale Durchführung der Experimente vertretbar, stellt jedoch bewusst keine vollständige Sicherheitsisolation dar und wäre für einen unbeaufsichtigten Produktivbetrieb entsprechend abzusichern [7, 25].

Ein zentrales Element der Integration ist die Behandlung von Fehlern, die aufgrund der probabilistischen Natur der Modelle und der Nutzung externer Dienste unvermeidlich auftreten. Dabei wird bewusst zwischen zwei Arten von Fehlern unterschieden: Vorübergehende Störungen des Dienstes, die sich etwa in bestimmten Statuscodes oder Verbindungsabbrüchen äußern, werden als transient eingestuft und der betreffende Aufruf mit exponentiell wachsender Wartezeit mehrfach wiederholt. Ein inhaltlich

fehlerhaftes, aber technisch wohlgeformtes Ergebnis, etwa ein Skript, das mit einem Programmfehler abbricht, wird hingegen nicht wiederholt, da es einen gültigen Messwert über die Fähigkeit des Modells darstellt und eine Wiederholung das Ergebnis verfälschen würde [6, 7]. Ergänzend werden anbieterspezifische Besonderheiten aktueller Modelle abgefangen: Lehnt ein Modell einen Sampling-Parameter wie die Temperatur ab, wird dieser automatisch weggelassen und die Anpassung im Ergebnisdatensatz vermerkt, sodass solche Abweichungen bei der Interpretation der Reproduzierbarkeit sichtbar bleiben. Für jeden Aufruf werden schließlich sämtliche relevanten Angaben, von der Modellkennung über den Token-Verbrauch und die Kosten bis hin zum erzeugten Code und dem Ausführungsergebnis, vollständig protokolliert und bilden so die lückenlose Grundlage für die anschließende Auswertung.

Der in Abschnitt 4.1 eingeführte zweite Ausführungsmodus, die direkte In-Context-Verarbeitung, ist über dieselbe einheitliche Modellschnittstelle umgesetzt, unterscheidet sich jedoch im Ablauf. Hier werden die Eingabedaten unmittelbar in den Prompt eingebettet und das Modell angewiesen, die transformierte Ergebnistabelle direkt in einem klar abgegrenzten Format zurückzugeben, das anschließend eingelesen und gegen die Soll-Lösung geprüft wird. Die Auswertung der Antwort ist dabei bewusst robust gegenüber am Ausgabelimit abgeschnittenen Antworten gestaltet, sodass auch unvollständige Ergebnisse noch verarbeitet und als solche erkannt werden. Da aktuelle Modelle standardmäßig einen internen Denkprozess ausführen, der das für die Ergebnistabelle verfügbare Ausgabebudget aufzehren würde, bevor die eigentlichen Daten erzeugt werden, wird dieses Verhalten für den Direkt-Modus deaktiviert und das Ausgabelimit angehoben [6, 24]. Anders als bei der wiederholten Code-Erzeugung wird jede Konfiguration in diesem Modus nur einmal ausgeführt, um den durch die eingebetteten Daten erhöhten Token-Verbrauch zu begrenzen; eine Wiederholung erfolgt ausschließlich bei einem technischen Fehlschlag wie einem transienten Anbieterfehler oder einer leeren beziehungsweise nicht auswertbaren Antwort, nicht hingegen bei einem wohlgeformten, aber inhaltlich abweichenden Ergebnis, das einen gültigen Messwert darstellt.

6.5 Umsetzung der Prompting-Strategien

Die in Abschnitt 4.4 konzipierten Strategien sind als versionierte Vorlagen umgesetzt, die getrennt vom Programmcode in einem eigenen, menschenlesbaren Format abgelegt sind. Jede Vorlage besteht aus einer eindeutigen Kennung, einer Systemanweisung und einer Nutzervorlage mit klar benannten Platzhaltern, in die zur Laufzeit die Auf-

gabenbeschreibung, die Ein- und Ausgabepfade sowie bei Bedarf das erwartete Zielschema eingesetzt werden. Diese versionierte und vom Code entkoppelte Ablage stellt sicher, dass jede eingesetzte Prompt-Variante eindeutig dokumentiert, unverändert wiederverwendbar und damit nachvollziehbar bleibt, was eine wesentliche Voraussetzung für die Vergleichbarkeit und Reproduzierbarkeit der Ergebnisse ist [23, 24].

Alle vier Strategien folgen einem einheitlichen Grundaufbau und unterscheiden sich ausschließlich im Umfang der zusätzlich bereitgestellten Hilfestellung, sodass beobachtete Qualitätsunterschiede eindeutig auf die jeweilige Strategie und nicht auf abweichende Formulierungen zurückführbar sind. Die Zero-Shot-Vorlage beschränkt sich auf die reine Aufgabenanweisung und bildet damit die Referenzbasis. Die Few-Shot-Vorlage ergänzt ein vollständig ausformuliertes Ein- und Ausgabepaar; dieses Beispiel bezieht sich bewusst auf eine andersartige Aufgabe auf einer separaten Beispieltabelle und nicht auf eine der neun evaluierten Aufgaben, damit es keine Musterlösung vorwegnimmt, sondern lediglich Stil und Format der erwarteten Lösung vermittelt [19, 23]. Die Chain-of-Thought-Vorlage leitet das Modell an, seine Lösung in nachvollziehbaren Zwischenschritten zu entwickeln, während die schema-angereicherte Vorlage der Anweisung eine explizite Beschreibung des Zielschemas mit Spaltennamen und Datentypen beifügt, um die strukturelle Korrektheit der Ausgabe gezielt zu unterstützen [22, 23]. Da das Zielschema nur von der schema-angereicherten Variante tatsächlich genutzt wird, während die übrigen Platzhalter für alle Strategien identisch belegt sind, bleibt der Vergleich über die Strategien hinweg fair und kontrolliert.

6.6 Mess- und Auswertungsverfahren

Die in Abschnitt 4.5 definierten Kriterien werden durch eine automatisierte Bewertungsroutine operationalisiert, die die erzeugte Ausgabedatei mit der bekannten Soll-Lösung abgleicht. Um zu berücksichtigen, dass programmatisch erzeugte Ergebnisse korrekte Werte häufig in abweichender Zeilenreihenfolge liefern, werden beide Tabellen vor dem Abgleich einheitlich sortiert und damit reihenfolgeunabhängig verglichen. Die Genauigkeit wird als Anteil der zellweise übereinstimmenden Werte bestimmt, die Vollständigkeit aus dem Vorhandensein der erwarteten Spalten und der korrekten Datensatzanzahl abgeleitet und die Konsistenz über die Übereinstimmung der Datentypen mit dem Zielschema erfasst [10, 12]. Ein zellweiser Abgleich setzt dabei voraus, dass Spaltenmenge und Datensatzanzahl übereinstimmen; ist diese strukturelle Grundlage nicht gegeben, wird dies gesondert ausgewiesen, statt eine inhaltliche Genauigkeit vorzutäuschen, die mangels vergleichbarer Struktur nicht sin-

nvoll bestimmbar wäre. Die Effizienzkennzahlen aus Zeit, Token-Verbrauch und Kosten stammen unmittelbar aus der standardisierten Modellantwort und werden ergänzend zur inhaltlichen Bewertung erfasst [20, 24].

Jeder einzelne Durchlauf wird als eigenständiger, unveränderlicher Ergebnisdatensatz abgelegt, der die vollständige Konfiguration, die erhobenen Qualitäts- und Effizienzkennzahlen, die exakte Modellkennung sowie den erzeugten Code umfasst. Diese lückenlose Protokollierung erlaubt es, jede spätere Auswertung ohne erneute Modelldaufrufe aus den gespeicherten Datensätzen nachzuvollziehen, und bildet zugleich die Grundlage für die Erhebung der Reproduzierbarkeit, die sich aus der Streuung der Kennzahlen über die wiederholten Ausführungen einer Konfiguration ergibt [6, 7]. Eine gesonderte Auswertungsschicht führt die einzelnen Datensätze zusammen, wählt je Konfiguration den jeweils jüngsten Lauf aus und klassifiziert die Ergebnisse nach ihrem Zustand, um technische Fehlschläge von inhaltlichen Abweichungen zu trennen. Auf dieser Grundlage werden die in Kapitel 8 dargestellten Vergleiche, Übersichten und detaillierten Abweichungsanalysen erstellt. Damit ist die technische Umsetzung des Untersuchungsdesigns abgeschlossen, und die tatsächliche Durchführung der Experimente sowie die dabei erhobenen Messdaten sind Gegenstand des folgenden Kapitels 7.

7. Durchführung der Experimente

Nachdem Kapitel 6 die technische Umsetzung des Untersuchungssystems beschrieben hat, dokumentiert dieses Kapitel die eigentliche Durchführung der Experimente. Es legt offen, in welchem Ablauf die einzelnen Läufe erfolgten (Abschnitt 7.1), welche Messdaten dabei in welcher Form erhoben wurden (Abschnitt 7.2) und unter welchen Bedingungen die Durchführung stattfand, um die Nachvollziehbarkeit und Reproduzierbarkeit der Ergebnisse sicherzustellen (Abschnitt 7.3). Damit schafft es die Grundlage für die anschließende Auswertung in Kapitel 8.

7.1 Versuchsablauf

Ausgangspunkt jeder Durchführung ist der in Kapitel 5 beschriebene synthetische Datensatz, der unverändert für alle Läufe verwendet wird, sowie die zu jeder Aufgabe vorab bestimmte Soll-Lösung. Auf dieser gemeinsamen Grundlage werden die Experimente vollständig automatisiert über den in Abschnitt 6.1 dargestellten Experiment-

Runner ausgeführt, sodass jede Konfiguration unter identischen Bedingungen und ohne manuelle Eingriffe durchläuft. Der Ablauf durchmustert die vollständige Faktormatrix systematisch, indem er für jede Kombination aus Modell, Prompting-Strategie und Aufgabe nacheinander die vorgesehenen Wiederholungen ausführt und das Ergebnis unmittelbar festhält [6, 10].

Entsprechend den beiden in Abschnitt 4.1 eingeführten Ausführungsmodi gliedert sich die Durchführung in mehrere Teilläufe. Im primären Modus der Code-Generierung wird für jede Konfiguration ein Verarbeitungsskript erzeugt, ausgeführt und bewertet; daraus ergeben sich bei vier Modellen, vier Prompting-Strategien, neun Aufgaben und je drei Wiederholungen insgesamt 432 Einzelläufe. Im ergänzenden Direkt-Modus wird jede der neun Aufgaben von jedem der vier Modelle einmalig unmittelbar auf den eingebetteten Daten gelöst, woraus sich 36 weitere Läufe ergeben. Zusätzlich löst die regelbasierte Vergleichspipeline dieselben neun Aufgaben in je einem deterministischen Durchlauf. Jeder einzelne Lauf wird direkt nach seiner Ausführung als eigenständiger Ergebnisdatensatz gespeichert, sodass der Gesamtablauf jederzeit unterbrochen und fortgesetzt werden kann, ohne dass bereits erhobene Ergebnisse verloren gehen oder überschrieben werden. Die auf den externen Modellzugriff zurückzuführenden, vorübergehenden Störungen werden dabei, wie in Abschnitt 6.4 beschrieben, durch eine automatische Wiederholung transienter Fehlaufäufe abgefangen, während inhaltlich fehlerhafte, aber technisch vollständige Ergebnisse bewusst als gültige Messwerte erhalten bleiben [6, 7].

7.2 Erhobene Messdaten

Für jeden Lauf wird ein umfassender Ergebnisdatensatz erhoben, der eine spätere Auswertung ohne erneute Modellaufäufe ermöglicht. Er umfasst zum einen die vollständige Konfiguration des Laufs, also das eingesetzte Modell mit seiner exakten Versionskennung, die verwendete Prompting-Strategie, die bearbeitete Aufgabe mit Kategorie und Schwierigkeitsgrad, den zugrunde liegenden Zufallskern, die Nummer der Wiederholung sowie die gewählte Temperatur. Zum anderen enthält er die Ergebnisse der in Abschnitt 4.5 definierten Bewertung, namentlich die Genauigkeit, die Vollständigkeit und die Konsistenz sowie die zugehörigen strukturellen Kennzeichen wie die Übereinstimmung von Schema und Datensatzanzahl, anhand derer die grundsätzliche Vergleichbarkeit eines Ergebnisses beurteilt wird [10, 12].

Ergänzend zu den Qualitätskennzahlen werden für jeden Lauf die Effizienzgrößen aus

Abschnitt 4.5.4 festgehalten, also der Verbrauch an Eingabe- und Ausgabe-Token, die daraus abgeleiteten Nutzungskosten sowie die Antwortzeit des Modells und, im Modus der Code-Generierung, die Dauer und der Erfolg der anschließenden Ausführung [20, 24]. Um über die reinen Kennzahlen hinaus auch die konkrete Lösung nachvollziehbar zu machen, wird zusätzlich das jeweils erzeugte Artefakt gesichert, im Code-Generierungs-Modus der vollständige erzeugte Quelltext und im Direkt-Modus die unmittelbar zurückgegebene Ergebnistabelle. Sämtliche Läufe werden in einem einheitlichen, maschinenlesbaren Format als jeweils unveränderlicher Datensatz abgelegt, während die erzeugten Ergebnistabellen im typerhaltenden Parquet-Format gespeichert werden. Insgesamt entsteht so aus den 432 Läufen der Code-Generierung, den 36 Läufen des Direkt-Modus und den neun Durchläufen der regelbasierten Pipeline eine geschlossene Datenbasis, auf der die gesamte Auswertung in Kapitel 8 aufsetzt.

7.3 Reproduzierbarkeit und Versuchsbedingungen

Die Reproduzierbarkeit wurde bereits im Untersuchungsdesign als querschnittliche Dimension verankert (Abschnitt 4.5) und schlägt sich unmittelbar in den Versuchsbedingungen nieder. Um vergleichbare Bedingungen zu schaffen, werden alle steuerbaren Einflussgrößen über sämtliche Läufe hinweg konstant gehalten: Der Datensatz wird über einen festen Zufallskern reproduzierbar erzeugt und unverändert verwendet, und die Temperatur wird durchgehend auf einen niedrigen Wert festgelegt, um die nicht durch das Modell selbst bedingte Variabilität zu minimieren [6, 23]. Da sich das Feld der Modelle mit hoher Geschwindigkeit weiterentwickelt und die genaue Modellfassung bei den kommerziell betriebenen Anbietern nicht selbst kontrolliert werden kann, wird die exakte Versionskennung jedes Modells bei jedem Aufruf protokolliert; im Rahmen dieser Arbeit kamen je ein aktueller Vertreter von Anthropic, OpenAI und Google sowie ein lokal ausgeführtes Modell der Llama-Reihe zum Einsatz [6, 20].

Um die probabilistische Variabilität der Modelle nicht nur zu kontrollieren, sondern selbst messbar zu machen, wird im primären Ausführungsmodus jede Konfiguration dreifach ausgeführt, sodass sich die Streuung der Ergebnisse über die Wiederholungen als Maß für die Stabilität auswerten lässt und damit die zur Reproduzierbarkeit formulierte Hypothese H3 überprüfbar wird [6, 7]. Der ergänzende Direkt-Modus wird demgegenüber bewusst nur einfach ausgeführt, um den durch die eingebetteten Daten deutlich erhöhten Token-Verbrauch zu begrenzen; die dort erhobenen Werte sind entsprechend als einzelne Beobachtung und nicht als über Wiederholungen gemittelte Größe zu ver-

stehen. Zu berücksichtigen ist ferner, dass einige aktuelle Modelle bestimmte Sampling-Parameter nicht mehr akzeptieren oder standardmäßig einen internen Denkprozess ausführen; solche anbieterspezifischen Abweichungen werden, wie in Abschnitt 6.4 beschrieben, abgefangen und im Ergebnisdatensatz vermerkt, sodass sie bei der Interpretation sichtbar bleiben. Das lokal betriebene Modell wurde auf allgemein verfügbarer Endgeräte-Hardware mit begrenztem Grafikspeicher ausgeführt, was insbesondere im Direkt-Modus eine Begrenzung des nutzbaren Kontextfensters erforderte und die praktischen Grenzen eines lokalen Betriebs sichtbar macht. Trotz dieser Vorkehrungen bleibt eine vollständige, bitgenaue Reproduktion aufgrund der probabilistischen Natur der Modelle und möglicher serverseitiger Änderungen der kommerziellen Modelle nicht garantierbar; die protokollierten Versionskennungen, die konstant gehaltenen Bedingungen und die vollständige Ablage aller Läufe stellen jedoch eine im Rahmen des Möglichen weitgehende Nachvollziehbarkeit sicher [6, 7].

8. Ergebnisse und Auswertung

8.1 Ergebnisse je ETL-Aufgabe

8.2 Vergleich der Modelle

8.3 Vergleich der Prompting-Strategien

8.4 Vergleich KI-basiert vs. klassisch

8.5 Diskussion der Ergebnisse

8.5.1 Stärken des LLM-Einsatzes

8.5.2 Schwächen und Risiken

8.5.3 Einordnung in den Stand der Forschung

9. Handlungsempfehlungen für die Praxis

9.1 Eignung von LLMs für unterschiedliche ETL-Szenarien

9.2 Empfehlungen zur Modell- und Prompt-Auswahl

9.3 Hybride Architekturen

10. Fazit und Ausblick

10.1 Zusammenfassung der Ergebnisse

DHBW

Luis Gerger

Page | 42

10.2 Beantwortung der Forschungsfragen

Bibliography

- [1] Bilal Khan et al. "An Overview of ETL Techniques, Tools, Processes and Evaluations in Data Warehousing". In: *Journal on Big Data* 6 (2024), pp. 1–20. ISSN: 25790048. DOI: 10.32604/jbd.2023.046223. URL: <https://www.proquest.com/docview/3200128360/abstract/ABA55C04CFAC48B2PQ/1> (visited on 05/08/2026).
- [2] Afef Walha, Faiza Ghozzi, and Faiez Gargouri. "Data Integration from Traditional to Big Data: Main Features and Comparisons of ETL Approaches". In: *The Journal of Supercomputing* 80.19 (Dec. 1, 2024), pp. 26687–26725. ISSN: 1573-0484. DOI: 10.1007/s11227-024-06413-1. URL: <https://doi.org/10.1007/s11227-024-06413-1> (visited on 05/08/2026).
- [3] Sarbaree Mishra. "Automating the Data Integration and ETL Pipelines through Machine Learning to Handle Massive Datasets in the Enterprise". In: *International Journal of Emerging Research in Engineering and Technology* 1.2 (June 30, 2020), pp. 69–78. DOI: 10.63282/3050-922X.IJERET-V1I2P109. URL: <https://ijeret.org/index.php/ijeret/article/view/231> (visited on 05/08/2026).
- [4] Kushvanth Chowdary Nagabhyru. *Bridging Traditional ETL Pipelines with AI Enhanced Data Workflows: Foundations of Intelligent Automation in Data Engineering*. Nov. 24, 2022. DOI: 10.2139/ssrn.5505199. Social Science Research Network: 5505199. URL: <https://papers.ssrn.com/abstract=5505199> (visited on 05/08/2026). Pre-published.
- [5] Chandran Ravi. "ETL (Extract, Transform & Load) Automation". In: *International Journal of Emerging Trends in Computer Science and Information Technology* 6.1 (Feb. 12, 2025), pp. 52–55. ISSN: 3050-9246. DOI: 10.63282/3050-9246.IJETCSIT-V6I1P106. URL: <https://www.ijetcsit.org/index.php/ijetcsit/article/view/37> (visited on 05/08/2026).
- [6] Wayne Xin Zhao et al. *A Survey of Large Language Models*. Mar. 18, 2026. DOI: 10.48550/arXiv.2303.18223. arXiv: 2303.18223 [cs]. URL: <http://arxiv.org/abs/2303.18223> (visited on 05/08/2026). Pre-published.
- [7] Hossein Hassani and Emmanuel Sirmal Silva. "The Role of ChatGPT in Data Science: How AI-Assisted Conversational Interfaces Are Revolutionizing the Field". In: *Big Data and Cognitive Computing* 7.2 (June 2023), p. 62. ISSN: 2504-2289. DOI: 10.3390/bdcc7020062. URL: <https://www.mdpi.com/2504-2289/7/2/62> (visited on 05/08/2026).

-
- [8] Soumen Chakraborty. "Beyond ETL: How AI Agents Are Building Self-Healing Data Pipelines". In: *Journal of Computer Science and Technology Studies* 7.3 (May 8, 2025), pp. 741–756. ISSN: 2709-104X. DOI: 10.32996/jcsts.2025.7.3.81. URL: <https://al-kindipublishers.org/index.php/jcsts/article/view/9427> (visited on 05/08/2026).
- [9] Jahangir Khan. "Leveraging Artificial Intelligence to Automate ETL Pipelines: Evolving Legacy Data Systems into Intelligent Workflows". In: (June 9, 2025).
- [10] Chaimae Boulahia et al. "The Multi-Criteria Evaluation of Research Efforts Based on ETL Software: From Business Intelligence Approach to Big Data and Semantic Approaches". In: *Evolutionary Intelligence* 17.4 (Aug. 1, 2024), pp. 2099–2124. ISSN: 1864-5917. DOI: 10.1007/s12065-023-00899-z. URL: <https://doi.org/10.1007/s12065-023-00899-z> (visited on 05/08/2026).
- [11] Sandeep Rangineni et al. "A Review on Enhancing Data Quality for Optimal Data Analytics Performance". In: *International Journal of Computer Sciences and Engineering* 11 (Oct. 31, 2023), pp. 51–58. DOI: 10.26438/ijcse/v11i10.5158.
- [12] Fakhitah Ridzuan and Wan Mohd Nazmee Wan Zainon. "A Review on Data Quality Dimensions for Big Data". In: *Procedia Computer Science*. Seventh Information Systems International Conference (ISICO 2023) 234 (Jan. 1, 2024), pp. 341–348. ISSN: 1877-0509. DOI: 10.1016/j.procs.2024.03.008. URL: <https://www.sciencedirect.com/science/article/pii/S187705092400365X> (visited on 05/08/2026).
- [13] Pierre-Olivier Côté et al. "Data Cleaning and Machine Learning: A Systematic Literature Review". In: *Automated Software Engineering* 31.2 (June 11, 2024), p. 54. ISSN: 1573-7535. DOI: 10.1007/s10515-024-00453-w. URL: <https://doi.org/10.1007/s10515-024-00453-w> (visited on 05/08/2026).
- [14] Raghavender Maddali. "Automating Data Quality Assurance Using Machine Learning in ETL Pipelines". In: *IJLRP - International Journal of Leading Research Publication* 2.6 (o.J.). ISSN: 2582-8010. DOI: 10.5281/zenodo.15107533. URL: <https://www.ijlrp.com/research-paper.php?id=1455> (visited on 05/08/2026).
- [15] Muhammad Fakhurul Safitra et al. "Advancements in Artificial Intelligence and Data Science: Models, Applications, and Challenges". In: *Procedia Computer Science*. Seventh Information Systems International Conference (ISICO 2023) 234 (Jan. 1, 2024), pp. 381–388. ISSN: 1877-0509. DOI: 10.1016/j.procs.2024.03.018. URL: <https://www.sciencedirect.com/science/article/pii/S1877050924003752> (visited on 05/08/2026).

-
- [16] Jasmin Praful Bharadiya. "The Role of Machine Learning in Transforming Business Intelligence". In: *International Journal of Computing and Artificial Intelligence* 4.1 (Jan. 1, 2023), pp. 16–24. ISSN: 27076571, 2707658X. DOI: 10.33545/27076571.2023.v4.i1a.60. URL: <https://www.computersciencejournals.com/ijcai/archives/2023.v4.i1.A.60> (visited on 05/08/2026).
- [17] Alhassan Mumuni and Fuseini Mumuni. "Automated Data Processing and Feature Engineering for Deep Learning and Big Data Applications: A Survey". In: *Journal of Information and Intelligence* 3.2 (Mar. 1, 2025), pp. 113–153. ISSN: 2949-7159. DOI: 10.1016/j.jiixd.2024.01.002. URL: <https://www.sciencedirect.com/science/article/pii/S2949715924000027> (visited on 05/08/2026).
- [18] Zijing Liang et al. "A Survey of Multimodal Large Language Models". In: *Proceedings of the 3rd International Conference on Computer, Artificial Intelligence and Control Engineering*. CAICE '24. New York, NY, USA: Association for Computing Machinery, Aug. 6, 2024, pp. 405–409. ISBN: 979-8-4007-1694-2. DOI: 10.1145/3672758.3672824. URL: <https://dl.acm.org/doi/10.1145/3672758.3672824> (visited on 05/08/2026).
- [19] Pengfei Liu et al. "Pre-Train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing". In: *ACM Comput. Surv.* 55.9 (Jan. 16, 2023), 195:1–195:35. ISSN: 0360-0300. DOI: 10.1145/3560815. URL: <https://dl.acm.org/doi/10.1145/3560815> (visited on 05/08/2026).
- [20] *The 2026 AI Index Report | Stanford HAI*. o.J. URL: <https://hai.stanford.edu/ai-index/2026-ai-index-report> (visited on 05/08/2026).
- [21] *Comparison of AI Models across Intelligence, Performance, and Price*. o.J. URL: <https://artificialanalysis.ai/models> (visited on 05/08/2026).
- [22] Pranab Sahoo et al. "A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications". In: (o.J.).
- [23] Sander Schulhoff et al. *The Prompt Report: A Systematic Survey of Prompt Engineering Techniques*. Feb. 26, 2025. DOI: 10.48550/arXiv.2406.06608. arXiv: 2406.06608 [cs]. URL: <http://arxiv.org/abs/2406.06608> (visited on 05/08/2026). Pre-published.
- [24] Kaiyan Chang et al. *Efficient Prompting Methods for Large Language Models: A Survey*. Dec. 2, 2024. DOI: 10.48550/arXiv.2404.01077. arXiv: 2404.01077 [cs]. URL: <http://arxiv.org/abs/2404.01077> (visited on 05/08/2026). Pre-published.

-
- [25] Jahangir Khan. *Cognitive ETL: Using AI to Interpret Complex Data Relationships*. Nov. 10, 2025. DOI: 10.2139/ssrn.5728984. Social Science Research Network: 5728984. URL: <https://papers.ssrn.com/abstract=5728984> (visited on 05/08/2026). Pre-published.
- [26] Naveen Reddy Singi Reddy and Mahitha Adapa. "AI-Driven Data Integration: Transforming Enterprise Data Pipelines through Machine Learning". In: *Journal of Computer Science and Technology Studies* 7.12 (Nov. 26, 2025), pp. 110–119. ISSN: 2709-104X. DOI: 10.32996/jcsts.2025.7.12.16. URL: <https://al-kindipublishers.org/index.php/jcsts/article/view/11533> (visited on 05/08/2026).
- [27] Lin Long et al. "On LLMs-Driven Synthetic Data Generation, Curation, and Evaluation: A Survey". In: *Findings of the Association for Computational Linguistics: ACL 2024*. Findings 2024. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 11065–11082. DOI: 10.18653/v1/2024.findings-acl.658. URL: <https://aclanthology.org/2024.findings-acl.658/> (visited on 05/08/2026).
- [28] Hsin-Yu Chang et al. *A Survey of Data Synthesis Approaches*. July 4, 2024. DOI: 10.48550/arXiv.2407.03672. arXiv: 2407.03672 [cs]. URL: <http://arxiv.org/abs/2407.03672> (visited on 05/08/2026). Pre-published.
- [29] Zhuoyan Li et al. "Synthetic Data Generation with Large Language Models for Text Classification: Potential and Limitations". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. EMNLP 2023. Ed. by Houda Bouamor, Juan Pino, and Kalika Bali. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 10443–10461. DOI: 10.18653/v1/2023.emnlp-main.647. URL: <https://aclanthology.org/2023.emnlp-main.647/> (visited on 05/08/2026).
- [30] Joao Fonseca and Fernando Bacao. "Tabular and Latent Space Synthetic Data Generation: A Literature Review". In: *Journal of Big Data* 10.1 (July 10, 2023), p. 115. ISSN: 2196-1115. DOI: 10.1186/s40537-023-00792-7. URL: <https://doi.org/10.1186/s40537-023-00792-7> (visited on 05/08/2026).
- [31] Mandeep Goyal and Qusay H. Mahmoud. "A Systematic Review of Synthetic Data Generation Techniques Using Generative AI". In: *Electronics* 13.17 (Jan. 2024), p. 3509. ISSN: 2079-9292. DOI: 10.3390/electronics13173509. URL: <https://www.mdpi.com/2079-9292/13/17/3509> (visited on 05/08/2026).

-
- [32] Janardhan Reddy Kasireddy. "The Role of AI in Modern Data Engineering: Automating ETL and Beyond". In: *ICT for Global Innovations and Solutions*. Ed. by Saurav Bhattacharya. Cham: Springer Nature Switzerland, 2026, pp. 667–693. ISBN: 978-3-032-02853-2. DOI: 10.1007/978-3-032-02853-2_47.
- [33] Raghav Maddali. "Autonomous AI Agents for Real-Time Data Transformation and ETL Automation". In: *INTERNATIONAL JOURNAL OF RESEARCH AND ANALYTICAL REVIEWS* 10 (July 23, 2023), p. 12. DOI: 10.5281/zenodo.15096256.
- [34] Jahangir Khan. *Intelligent Anomaly Detection in ETL Workflows Using AI*. Nov. 9, 2025. DOI: 10.2139/ssrn.5729002. Social Science Research Network: 5729002. URL: <https://papers.ssrn.com/abstract=5729002> (visited on 05/08/2026). Pre-published.
- [35] Chiara Van Der Putten. "Transforming Data Flow: Generative AI in ETL Pipeline Automatization". laurea. Politecnico di Torino, Apr. 11, 2024. 83 pp. URL: <https://webthesis.biblio.polito.it/30855/> (visited on 05/08/2026).
- [36] Shashank A. "AI-Enhanced ETL Processes: Leveraging Artificial Intelligence for Optimized Data Integration Systems". In: *Journal Of Multidisciplinary* 5.8 (Aug. 10, 2025), pp. 219–225. ISSN: 2945-3445. URL: <https://sarcouncil.com/2025/8/ai-enhanced-etl-processes-leveraging-artificial-intelligence-for-optimized-data-integration-systems> (visited on 05/08/2026).
- [37] Dhamotharan Seenivasan. *AI Driven Enhancement of ETL Workflows for Scalable and Efficient Cloud Data Engineering*. June 28, 2024. Social Science Research Network: 5153853. URL: <https://papers.ssrn.com/abstract=5153853> (visited on 05/08/2026). Pre-published.
- [38] Indu Joshi et al. "Synthetic Data in Human Analysis: A Survey". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.7 (July 2024), pp. 4957–4976. ISSN: 1939-3539. DOI: 10.1109/TPAMI.2024.3362821. URL: <https://ieeexplore.ieee.org/abstract/document/10423161> (visited on 05/08/2026).
- [39] Jing Lin and Kee Yuan Ngiam. "How Data Science and AI-based Technologies Impact Genomics". In: *Singapore Medical Journal* 64.1 (Jan. 2023), p. 59. ISSN: 0037-5675. DOI: 10.4103/singaporemedj.SMJ-2021-438. URL: https://journals.lww.com/smj/fulltext/2023/01000/How_data_science_and_AI_based_technologies_impact.10.aspx?context=LatestArticles (visited on 05/08/2026).